

SEC: Symmetric Encrypted Computation via Fast Look-ups

Debadrita Talapatra
IIT Kharagpur, India

Nimish Mishra
IIT Kharagpur, India

Arnab Bag
IMEC Belgium

Sikhar Patranabis
IBM Research India

Debdeep Mukhopadhyay
IIT Kharagpur, India

October 8, 2025

Abstract

Garbled Circuits (GCs) and Fully Homomorphic Encryption (FHE) are state-of-the-art general-purpose primitives allowing a client to securely outsource the storage and processing of sensitive private data to an untrusted third party cloud server. GCs provide data privacy and function privacy while being efficient, but suffer from non-reusability of its translation tables. On the other extreme, FHE enables arbitrary encrypted computation ensuring data-privacy but suffers from huge computation overhead and limited scalability. Here, we explore the following question- *Can encrypted computation guaranteeing data privacy be enabled through use of encrypted lookup tables (like GC) that are reusable for practically large circuits?* Such a solution will retain the efficiency of GCs (because of encrypted table lookups); and while it cannot achieve unbounded reusability (like FHE, because of the inclination on computation rather than lookups), an interesting question is whether its reusability bound can be made sufficient for *practically large* circuits.

In this work, we propose Symmetric Encrypted Computation (SEC), the first practically efficient and provably secure lookup-based construction that supports *reusable* evaluations of arbitrary Boolean circuits over symmetrically encrypted data, while ensuring data-privacy guarantees. With a single setup of encrypted look-up tables, SEC supports $O(k)$ *re-evaluations* of the *same* circuit of size n (for client controlled $k (\geq n)$). Such reusability improves upon GC and is asymptotically large enough to scale to practical deployments of realistic circuits; also note that SEC completely bypasses bootstrapping, thereby being more efficient than FHE. This is achieved by relying on purely symmetric-key crypto-primitives by extending and generalizing the functional capabilities of Searchable Symmetric Encryption (SSE), while inheriting its desirable performance benefits. Reusability through SSE without leakage is ensured in SEC by the design choice to treat a generic 2-input Boolean gate as a 3-input encrypted query, with the third input randomizing access patterns across reuse.

We provide a concrete construction of SEC and analyze its security with respect to a rigorous leakage profile. We also experimentally validate its practical efficiency. SEC outperforms state-of-the-art FHE schemes (such as Torus FHE) substantially. We further showcase the scalability of SEC for functions with multi-bit inputs via experiments performing encrypted evaluation of the entire AES-128 circuit, as well as the max-pooling layers of AlexNet architecture. For both sets of experiments, SEC outperforms state-of-the-art unaccelerated FHE implementations by $1000\times$ in terms of processing time, while incurring $250\times$ lower storage.

Contents

1	Introduction	3
1.1	Our Contributions	4
1.2	Technical Overview	7
1.3	Related Work	13
1.4	How SEC differs from GC and HE	14
2	Preliminaries and Background	16
2.1	Conjunctive SSE: Syntax and Security Model	16
2.2	Adaptive Security of CSSE	17
2.3	Oblivious Cross-Tag Protocol (OXT): Overview	18
3	Symmetric Encrypted Computation	19
3.1	Syntax of SEC	19
3.2	SEC Construction	20
4	Proof of Correctness of SEC	24
5	Complexity Analysis of SEC	25
6	Leakage Profile Description and Security Analysis	25
6.1	Leakages inherited (alt. rendered inconsequential) from OXT	26
6.2	Security Analysis of SEC_{OXT}	27
7	Statistical Analysis of Leakage Due to Reusability	31
8	Experimental Results	32
9	Discussion	34

1 Introduction

Outsourced Storage. The upswing in data production in today’s digitally-driven world has motivated the concept of outsourcing data to third-party cloud servers for storage. However, such *outsourced storage* solutions are often plagued by security breaches that lead to disclosure of client data [23, 52, 53]. Without specific privacy mechanisms, third-party cloud servers could gain access to sensitive user data, thus leading to serious privacy concerns. This establishes the requirement of adopting secure and scalable privacy mechanisms for protecting sensitive outsourced data from unauthorized access. A straightforward solution to this problem is to encrypt this data before offloading to the third-party server, thereby ensuring data privacy and preventing disclosures. However, this leads to the challenge of securely computing queries (or more generally, executing functions/programs) directly on the encrypted data without decrypting it first.

Computation on Encrypted Outsourced Data. In the setting of encrypted outsourced storage, an integral concern is, *can we design privacy-preserving mechanisms that enable arbitrary computation over encrypted data without requiring explicit decryption?* This problem of privacy-preserving computation on encrypted outsourced data has been extensively studied, yielding several powerful paradigms, differing in adversarial models, communication assumptions, and security guarantees. We brief them now.

Garbled Circuits (GCs) [5, 34, 40] enable secure circuit evaluation via encrypted table lookups, maintaining privacy of the inputs, outputs, and of the entire circuit. However, their non-reusability necessitates refreshing translation tables post each computation. In contrast, Multi-party Computation (MPC) instead distributes client data among multiple servers, guaranteeing privacy as long as the adversary corrupts no more than a threshold number of parties. Certain MPC protocols leverage GCs to hide circuits when evaluated on a single input. Efficient implementations of MPC and GCs exist [6, 36, 46, 49, 50], particularly in the honest-majority setting where only a minority of parties are corrupted. Likewise, Fully Homomorphic Encryption (FHE) [12, 27, 33] supports evaluation of arbitrary (polynomial-size) circuits directly on ciphertexts in the single-client, single-server model, ensuring that no information is leaked without the secret key. Similar to FHE, Leveled Homomorphic Encryption (LHE) [11, 13, 35, 37, 39] improves efficiency by enabling evaluation of bounded-depth circuits without bootstrapping, but supports only a limited number of multiplicative levels (typically 10–100), beyond which excessive noise causes decryption failures. Finally, traditional Functional Encryption (FE) [9, 43] allows more fine-grained query evaluation, leaking only the output of the computation to the server. Despite significant optimizations in recent years [1, 2, 3, 8, 10, 19, 20, 21, 25, 28, 29, 30, 31, 32, 38], both FHE and FE remain computationally expensive and do not scale well to large datasets.

Encrypted Computation Framework. In a single-client, single-server setting, encrypted computation mechanisms can be broadly classified into two categories: those relying on table lookups (e.g., GCs), and those based on explicit circuit evaluation (e.g., FHE). Each approach offers distinct advantages and limitations. FHE provides a strong and ideal notion of security, but circuit evaluation is computationally expensive. As circuit depth increases, ciphertext noise accumulates, necessitating an expensive noise-management technique known as bootstrapping. Despite significant progress in optimizing bootstrapping, FHE remains prohibitively expensive for practical large-scale deployment.

In contrast, GCs enable general-purpose circuit evaluation through encrypted table lookups, guaranteeing privacy of both inputs/outputs and the circuit itself. However, a fundamental limitation is their non-reusability, a garbled translation table can only be used once for a single circuit evaluation. Reusing the same tables across multiple evaluations leaks non-trivial

Properties	GC	LHE	SEC (This Work)	FHE
Reusability	One-time SFE	Bounded; $O(1)$ levels, $10 \sim 50$ [37, 39]	Bounded; $O(k)$ circuit re-evaluations	Unbounded
Bootstrapping	✗	✗	✗	✓
Computation Model	Lookup Table	Circuit Evaluation	Lookup Table	Circuit Evaluation
Interaction Round	Setup for every evaluation	One-time Setup	One-time Setup	One-time Setup
Privacy Guarantee	SFE + Data Privacy	Data Privacy	Data Privacy	Data Privacy
Security Assumption	Symmetric-key Encryption	Lattice-based	Symmetric-key Encryption	Lattice-based

Table 1: Comparison between state-of-the-art privacy-preserving encrypted frameworks with SEC. In the table GC: Garbled Circuit; LHE: Leveled Homomorphic Encryption; SEC: Symmetric Encrypted Computation; FHE - Fully Homomorphic Encryption; SFE: Secure Function Evaluation; n : size of the circuit, *controlled by client*. : gives a tradeoff between practical efficiency and optimal functionality; : Performance Bottleneck; : Optimal Functionality. By design, SEC supports $O(k)$ ($k \geq n$) re-evaluations for *same* circuit of size n with a single setup.

information and compromises input/output privacy. Kamara and Williams [40] further demonstrated that a garbled table can be interpreted as a two-dimensional encrypted array, where gate evaluation corresponds to querying such arrays, thereby establishing a connection between GCs and Structured Encryption (STE). Nonetheless, they inherit the same limitation of non-reusability, which we improve in this work.

Motivation. Despite significant advances, existing primitives expose a gap: GC-based approaches enable table look-ups but are inherently non-reusable; likewise, FHE provides strong security but remains prohibitively inefficient. As such, there is a pressing need for a primitive that simultaneously achieves strong security guarantees, practical efficiency, and reusability. In this paper, we take a step toward filling this gap by investigating whether arbitrary Boolean circuit evaluations over encrypted binary inputs can be supported through an efficient, reusable table look-up mechanism. Concretely, we ask:

Can we enable arbitrary Boolean circuit evaluation over encrypted data table look-ups, that are reusable for practically large circuits, with a single setup?

1.1 Our Contributions

In this work, we address the above problem of enabling efficient and reusable encrypted computation in the single-client, single-server model of outsourced computation¹. We introduce *Symmetric Encrypted Computation* (SEC), a novel framework for the practical evaluation of arbitrary Boolean functions over symmetrically encrypted data. At the core of SEC is a new methodology that maps Boolean gate evaluations over encrypted inputs to look-ups over encrypted tables. We demonstrate, for the first time, how techniques from Searchable Symmetric Encryption (SSE) [15, 18, 24, 15, 44], originally restricted to encrypted search operation only can be extended to support *arbitrary* Boolean circuit evaluations without requiring any special input structure. Unlike GC, SSE specific primitives allow SEC to guarantee data privacy through usage of encrypted lookup tables that are *reusable* across multiple evaluations of the same Boolean circuit. While the number of such reuses is bounded (see Table 1), we show that our design of SEC allows for sufficiently large reuse bound for practically large circuits. Therefore, even though SEC does not achieve the unbounded reuse that FHE has, it still caters to a wide majority of real-world circuits. In other words, for a circuit of size $O(n)$,

¹Our primary objective is to preserve the privacy of both the plaintext inputs and the outputs against a semi-honest adversarial server. Verifiable outsourced computation (like [26]) addresses malicious adversaries, but current constructions remain far from practical; hence, we restrict ourselves to the semi-honest setting.

SEC allows reusing the same encrypted tables for up to $O(k)$ evaluations of the *same* circuit²; each new circuit is allowed $O(k)$ fresh re-executions, irrespective of the number of executions already done for *other* circuits. This allows SEC to retain the efficiency of GC and avoid the computational bottlenecks (like bootstrapping) for FHE. Furthermore, we design SEC in a manner which offsets known leakage in SSE: SEC treats a generic 2-input Boolean gate as a 3-input encrypted query, with the third input randomizing access patterns across reuse. Such client-controlled randomization of a dummy third input to a Boolean gate in SEC prevents compromising data privacy through inherent SSE leakages. This leakage suppression furthermore becomes the foundation of enabling reusability, as we show later. To the best of our knowledge, SEC is the first provably secure and reusable framework that enables arbitrary function evaluation over encrypted data using efficient lookup-based techniques.

Structural Overview. SEC relies on “encoding” any arbitrary Boolean circuit as a series of lookup tables, where each table corresponds to a primitive Boolean gate. It then replaces explicit circuit computation via encrypted look-up operations by deploying a fast, encrypted search mechanism. Informally speaking, the design mechanism of SEC relies on the observation that any Boolean function can be represented by an equivalent logic circuit composed of Boolean variables and basic Boolean logic gates - $\{\text{XOR}, \text{AND}, \text{OR}\}$ ³. It involves creating the encrypted lookup tables for these basic gates to facilitate the evaluation of arbitrary Boolean functions. We then leverage, in a black-box way, existing SSE techniques for fast conjunctive look-ups over such encrypted lookup tables for the primitive operations (which we model as “encrypted search indices” as in standard SSE scheme. Refer to Section 2.1 for background on SSE). As it turns out, this enables extremely fast circuit evaluation matching the efficiency of GC and surpassing the performance of the most efficient FHE schemes by several orders of magnitude. We present two concrete instantiations of SEC based on two practically efficient conjunctive SSE constructions: Oblivious Cross Tags (OXT) [15] (Refer to Section 2.3 for overview of OXT) and CONJFILTER [44] (the former was the first practically efficient conjunctive SSE scheme to be proposed while plugging in non-trivial leakages, while latter supports particularly fast searches since it is based on purely symmetric-key cryptoprimitives). We call the resulting schemes SEC_{OXT} and $\text{SEC}_{\text{CONJFILTER}}$, respectively.

Supporting Arbitrary Boolean Functions. A pivotal feature of SEC is its ability to evaluate function compositions, which is the key to evaluating arbitrary circuits. An arbitrary Boolean function decomposes into an expression of function composition with a lesser number of variables (following Shannon’s theory, for Boolean circuits). Consequently, a complex function can be evaluated using SEC with optimal computation overhead proportional to the circuit size (See Table 8). For a circuit of size n , SEC can compute up to $O(k)$ ($k \geq n$) evaluations of the *same* circuit⁴ of the circuit with a single setup of the encrypted lookup tables, without leaking any correlation between multiple evaluations. As later sections elaborate, SEC treats a 2-input Boolean gate as a 3-input gate, with the third input being a dummy input specific to conjunctive SSE constructions. A client controlled randomization of this third input then suppresses SSE specific leakages observable by the semi-honest server for *re-executions* of the same gate (and by extension, the entire circuit), thereby bounding reusability for the *same* circuit by the number of iterative derangements possible in a set of $O(k)$ dummy inputs. Each new circuit is allowed $O(k)$ fresh re-executions, irrespective of the number of executions already done for *other* circuits. We stress that such reusability is possible because lookup tables in SEC are structured around

²For client controlled $k \geq n$. Higher k implies more evaluations, with added server storage cost (that is acceptable; see concrete storage in Tab. 2 and Sec. 8).

³ $\{\text{XOR}, \text{AND}\}$ is functionally complete, assuming a constant input (i.e. 1 or HIGH) is available. We provide support for 3 gates for convenience in computation.

⁴For client controlled $k \geq n$. Higher k implies more evaluations, with added server storage cost (that is acceptable; see concrete storage in Table 2 and Section 8).

state-of-the-art conjunctive SSE specific encrypted data structures; it is unclear whether such randomization is possible in state-of-the-art GC specific encrypted translation tables, and may require further investigation beyond the scope of this work.

Additionally, SEC incurs nominal storage overhead for encrypted lookup tables compared to the substantial storage required to store the bootstrapping key in FHE schemes (See Table 7). It also supports arbitrary function evaluations in a single round of communication with the expense of a small amount of additional storage while minimizing leakages compared to a multi-round solution. Collectively all these features render SEC practically ideal for encrypted outsourced computation frameworks.

Practical Instantiation of SEC. Our generic privacy-preserving computation framework SEC can be practically implemented by deploying any conjunctive SSE scheme as a black-box. We provide concrete constructions of SEC using two conjunctive SSE schemes:

SEC_{OXT}. We analyze a concrete instantiation of SEC based on the OXT protocol [15] (See Section 2.3 for a brief overview), abbreviated SEC_{OXT}. Our analysis fundamentally covers the complexity of SEC_{OXT} in terms of performance and storage overhead, and a formal security analysis based on a well-defined leakage profile. We provide a detailed complexity analysis of SEC_{OXT} based on our experimental results in Section 8. The leakage profile and security proof of SEC_{OXT} are elaborated in Section 6.

SEC_{CONJFILTER}. We also demonstrate the scalability and complexity overhead of a second instantiation of SEC by deploying a purely symmetric-key based (plausibly quantum-safe) conjunctive SSE, CONJFILTER [44]. We provide performance and storage overhead analysis of SEC_{CONJFILTER} in Section 8.

Security Analysis. SEC relies on an efficient, adaptively secure conjunctive SSE scheme for efficient encrypted lookup-based function evaluation. However, the inherent design of SEC averts the direct extrapolation of several non-trivial leakages of the underlying SSE scheme. This in turn enables SEC to prevent correlation between multiple circuit evaluations. *We emphasize, that the same look-up table can be used for $O(k)$ evaluations of the same circuit⁵ of an n -size Boolean circuit. When the circuit changes, the same setup can be reused for the new circuit another $O(k)$ evaluations, irrespective of how many times the previous circuit was executed.* The privacy of input/output data bits is ensured by the semantic security guarantee of the IND-CPA secure symmetric-key encryption using which the bits are encrypted. We present a detailed security and leakage profile analysis of SEC_{OXT} following the security properties of the underlying conjunctive SSE scheme OXT. We elucidate the improvements in leakage due to our improvised construction of the lookup tables in Section 6 and 6.2.

Performance And Scalability. We demonstrate the efficacy and scalability of SEC_{OXT} and SEC_{CONJFILTER} by evaluating basic Boolean gates and cascaded gates as function composition. Section 8 gives a detailed analysis of our experimental evaluations. SEC decomposes any arbitrary (> 2 number of input-bits) circuit into universal binary gates {XOR, AND, OR} and reuses this storage across multiple computations of the same gate, thus *preventing exponential storage-blowup*. In Section 8, we showcase scalability of SEC_{OXT} and SEC_{CONJFILTER} for functions with multi-bit inputs by using it for encrypted evaluation of the entire AES-128 circuit and three max-pooling layers of AlexNet architecture⁶. In Table 2 below, we show a practical use case of the AES SBox⁷, wherein SEC outperforms various TFHE [21] backends by orders of magnitude. For the AES SBox, we run the setup procedure for SEC with 50000 “special” terms, implying

⁵For client controlled $k \geq n$. Higher k implies more evaluations, with added server storage cost (that is acceptable; see concrete storage in Tab. 2 and Sec. 8).

⁶KSH17 Imagenet classification with deep convolutional neural networks

⁷For fair evaluation, we use *unparallelized* version of the SBox [42], which involves 5 XORs per bit in the output, thereby totaling 40 XORs for the entire byte.

SEC can execute the **same** AES SBox circuit 50000 times without requiring a fresh setup. With the same nominal storage overhead for 50000 “special” terms, several other circuits⁸ like 64-bit adder, subtractor, multiplier, division, floating point arithmetic, and others can be performed reliably, allowing up to 50000 reuses for a single circuit.

Table 2: Time taken (in seconds) and storage overhead (in MB) for evaluation of one byte AES SBox by SEC_{OXT} and SEC_{CONJFILTER} against different TFHE backends. Here, $k = 50000$, thus SEC allows up to $O(k) = 50000$ reuses of the *same* SBox circuit with a single setup (sufficiently large for several encryptions of AES).

Scheme	Time taken (in seconds)	Storage (in MB)
TFHE-Nayuki AVX	14.85	24
TFHE-Nayuki Portable	22.862	24
TFHE-Spqlios AVX	4.21	24
TFHE-Spqlios FMA	2.57	24
SEC _{CONJFILTER} (This work)	0.1013	0.814
SEC _{OXT} (This work)	0.96	0.106

1.2 Technical Overview

Efficient “search” over Encrypted Lookup Tables. The first step towards constructing SEC is to create efficient mechanisms for encrypted table lookups, for which we rely on Searchable Symmetric Encryption (SSE) [15, 24, 44, 45, 47] specifically supporting conjunctive queries [15, 44]. SSE offers a *restricted yet efficient* set of capabilities, it allows fast and efficient searches over symmetrically encrypted data. Although traditionally, SSE schemes have been used exclusively for searches, we show that SSE data structures are amenable to design modifications such that *efficient searches can be used to compute arbitrary Boolean functions on encrypted data*. Any function computation can be modeled as an equivalent logic circuit composed of Boolean variables and universal basic logic gate set {XOR, AND, OR}. Hence, ① creating encrypted lookup tables for these basic gates; ② translating these lookup tables to encrypted search indices over which a typical SSE scheme operates; and ③ using efficient *search* capabilities of SSE constructions supporting conjunctive queries over these encrypted indices, is equivalent to *computing* the logic gates over encrypted inputs.

Translating Lookup Table to Search Index. We explicate the foundational building blocks of the SEC framework with the help of a concrete example - by explaining the end-to-end evaluation of a XOR gate (AND and OR follow suit). Given *encryptions* of two plaintext bits x and y by the client to the server, SEC uses efficient search capabilities of a generic *conjunctive* SSE scheme, CSSE⁹, (See Section 2 for details) which returns a single *encrypted* output bit. Upon decryption on the client side, a plaintext bit 0/1 is obtained, that is equal to the actual value of XOR(x, y).

A generic SSE scheme, operates on an inverted index-style data structure, which maps every keyword in a database to all the documents in which it occurs. In order for SEC to compute basic binary gates using encrypted search capability of a conjunctive SSE scheme, it needs to translate the function tables of the binary gates into an inverted index of keywords and documents (tagged by document identifiers). Therefore, every plaintext bit (x or y) is *mapped* to some keyword $\mathbf{w}$ (alphanumeric strings). For example, when the plaintext input bit $x = 1$,

⁸Refer <https://nigelsmart.github.io/MPC-Circuits/> for exact circuit descriptions.

⁹Our analysis, proofs, and implementations are based on a CSSE scheme and we will use SSE, conjunctive SSE and CSSE interchangeably for our explanations henceforth.

Table 3: Contents of documents related to the functional evaluation of 2-bit XOR, as well as mapping of document identifiers to their corresponding keywords. Here, Enc_k refers to any generic symmetric encryption scheme with secret key k .

Input bit x	Keyword Map of x	Input bit y	Keyword Map of y	XOR(x, y)	doc_id	Doc. content
0	$\bar{\mathbf{w}}_1$	0	$\bar{\mathbf{w}}_2$	0	D_0	$Enc_k(0)$
0	$\bar{\mathbf{w}}_1$	1	$\mathbf{w}_2$	1	D_1	$Enc_k(1)$
1	$\mathbf{w}_1$	0	$\bar{\mathbf{w}}_2$	1	D_2	$Enc_k(1)$
1	$\mathbf{w}_1$	1	$\mathbf{w}_2$	0	D_3	$Enc_k(0)$

it is mapped to keyword $\mathbf{w}_1$; conversely, when $x = 0$, we map it to keyword $\bar{\mathbf{w}}_1$. Likewise, input bit y is mapped to $\mathbf{w}_2$ or $\bar{\mathbf{w}}_2$ depending on whether it is 1 or 0 respectively. The actual plaintext output of $XOR(x, y)$ is encrypted and stored as the document set indexed by identifiers $\text{doc_id } (D_i) : i \in \{0, 1, 2, 3\}$. For underlying SSE-based encrypted lookup in SEC, we populate the *inverted search index* representation (as shown in Table 4). The idea is to map a given keyword to the doc_ids which are related to the keyword (as shown in Table 3). The search index (improvised lookup table for XOR) is encrypted and offloaded to the server as done in a typical SSE scheme.

For computing a function, say, $XOR(x, y)$, keywords corresponding to encrypted inputs x and y are chosen and sent as a conjunctive query to the server. We note that *there is exactly one common document between any combination of “Keyword Map of x ” and “Keyword Map of y ”* (in Table 3) which is returned as a response to the underlying conjunctive SSE query. This design, in conjunction with the “Doc. content” from Table 3, ensures that only the correct evaluation of XOR is returned as a result of querying on SEC. It is crucial to note that at no point in this entire process have we performed an explicit computation of XOR gate (as done in FHE). The entire evaluation is completed by *searching* over encrypted lookup tables (search index) which is extremely fast and efficient.

“Dummy” Keywords/Documents. We add k “special” terms $\mathbf{w}_d^1, \dots, \mathbf{w}_d^k$ (See Table 4) for a circuit with n gates ($k \geq n$). “Special” terms are random alphanumeric strings (used as dummy keywords), that are mapped to all documents ($D_0, \dots, D_3$) that are present in the search index. It is also observed (in Table 4) that some dummy documents ($D'_0, D'_1 \dots$) are added to the search index to ensure that the frequency of “actual” keywords ($\mathbf{w}_1, \mathbf{w}_2, \bar{\mathbf{w}}_1, \bar{\mathbf{w}}_2$) matches that of the “special” terms ($\mathbf{w}_d^1, \dots, \mathbf{w}_d^k$), thereby ensuring a uniform frequency of all keywords in the final encrypted database¹⁰. These dummy documents comprise encryptions of unique random alphanumeric values. Note that this design choice of adding dummy keywords/documents does not have any impact on the correctness of the functional evaluation. The significance of this design choice is later reflected in reusing the same look-up table for computing the same/different circuits more than once without revealing any correlation for the adversary to infer between the computations.

SEC Workflow: Without loss of generality, assume evaluation of $XOR(1, 1)$ using SEC. This translates to a conjunctive query $q = \mathbf{w}_d^1 \wedge \mathbf{w}_1 \wedge \mathbf{w}_2$ (as inferred from Table 3), where $\mathbf{w}_d^1$ is randomly chosen from the set of k ($k \geq n$) “special” terms for one gate evaluation (i.e. for one conjunctive query). We explicate the sequence of operations that are consequently executed (assuming a black-box conjunctive SSE search), and also draw parallels with conjunctive SSE terminology used in literature below:

¹⁰This example is shown for circuits with XOR gate only. Similar strategy follows for AND and OR gates. Thus for any arbitrary circuit of depth n , we add k ($k \geq n$) dummy keywords for the entire database entailing all three logic gates.

Table 4: An inverted search index representation of the database (search index) for XOR function.

Keywords	doc_id
$\bar{\mathbf{w}}_1$	D_0, D_1, D'_0, D'_1
$\mathbf{w}_1$	D_2, D_3, D'_2, D'_3
$\bar{\mathbf{w}}_2$	D_0, D_2, D'_4, D'_5
$\mathbf{w}_2$	D_1, D_3, D'_6, D'_7
$\mathbf{w}_d^1$	D_0, D_1, D_2, D_3
$\mathbf{w}_d^2$	D_0, D_1, D_2, D_3
$\vdots$	$\vdots$
$\mathbf{w}_d^k$	D_0, D_1, D_2, D_3

1. **Processing “special” Term:** Use the first keyword in q (i.e. $\mathbf{w}_d^1$), generate an address, index into the encrypted lookup table, and retrieve an encrypted list of `doc_id` mapped to $\mathbf{w}_d^1$. For consistency, we denote $\text{sval}_{\mathbf{w}, D_j}$ as the retrieved output from this phase for some arbitrary keyword $\mathbf{w}$ and document D_j . In our example, this phase shall return the encrypted list $[\text{sval}_{\mathbf{w}_d^1, D_0}, \text{sval}_{\mathbf{w}_d^1, D_1}, \text{sval}_{\mathbf{w}_d^1, D_2}, \text{sval}_{\mathbf{w}_d^1, D_3}]$ (the encrypted entries corresponding to $\mathbf{w}_d^1$, see Table 4). The choice of the “special” term is dependent on the underlying SSE scheme; generally in OXT or CONJFILTER the “special” term is selected according to the least frequent keyword/conjunct in a given conjunctive query for efficiency purposes. In our instantiation of SEC_{OXT} and $\text{SEC}_{\text{CONJFILTER}}$, it is set to the first keyword/conjunct (since the frequency of all keywords is the same).
2. **Processing “actual” keywords.** In this phase, some auxiliary information dependent on the tuples $(\mathbf{w}_d^1, \mathbf{w}_1)$ and $(\mathbf{w}_d^1, \mathbf{w}_2)$ is used in conjunction with retrieved `sval` set to create a *search token*. For consistency, we denote $\text{Token}_{\mathbf{w}_d^1, \mathbf{w}_i, D_j}$ to denote a search token generated upon combination of $\text{sval}_{\mathbf{w}_d^1, D_j}$ and auxiliary information dependent on tuple $(\mathbf{w}_d^1, \mathbf{w}_i)$ ($\mathbf{w}_i$ refers to the actual keywords representing the encrypted input bits) in q , in this example $\mathbf{w}_i \in \{\mathbf{w}_1, \mathbf{w}_2\}$). Upon being indexed into SSE specific data structures, $\text{Token}_{\mathbf{w}_d^1, \mathbf{w}_i, D_j}$ returns a binary decision on whether keyword $\mathbf{w}_i$ is present in document D_j . In our example thus, for every $\text{sval}_{\mathbf{w}_d^1, D_j}$ in $[\text{sval}_{\mathbf{w}_d^1, D_0}, \text{sval}_{\mathbf{w}_d^1, D_1}, \text{sval}_{\mathbf{w}_d^1, D_2}, \text{sval}_{\mathbf{w}_d^1, D_3}]$, one token is generated for every keyword of the query other than $\mathbf{w}_d^1$ i.e., $\mathbf{w}_1$ and $\mathbf{w}_2$. We denote them by $\text{Token}_{\mathbf{w}_d^1, \mathbf{w}_1, D_j}$ and $\text{Token}_{\mathbf{w}_d^1, \mathbf{w}_2, D_j}$ respectively. These tokens are then indexed into SSE specific data structures to return a binary decision on whether both keywords (i.e. $\mathbf{w}_d^1$ and $\mathbf{w}_1/\mathbf{w}_2$) are present in `doc_id` D_j ¹¹.
3. **Query Result.** Any document identifier which is included in the output of step ①, and for which *all* search tokens generated in step ② give a positive binary result, is included in the *result* of the query. Note that, for our example, only document D_3 contains all the keywords $\mathbf{w}_d^1$, $\mathbf{w}_1$, and $\mathbf{w}_2$. Hence, the output of the query $q = (\mathbf{w}_d^1 \wedge \mathbf{w}_1 \wedge \mathbf{w}_2)$, for this example, is $\text{sval}_{\mathbf{w}_d^1, D_3}$. This result is returned back to the client, which upon decryption obtains D_3 . It follows trivially that since $D_3 = \text{Enc}_k(0)$ (see Table 3), query q and associated *search* over the encrypted lookup table (search index¹²) has effectively *computed* $\text{XOR}(1, 1)$.

Concretely, we design the encrypted search index in a way such that for arbitrary encrypted one-

¹¹This flow of token generation is specific to underlying SSE algorithm.

¹²The terminology “encrypted lookup table” and “encrypted search index” are used interchangeably henceforth.

bit inputs x and y (their corresponding keyword mappings), *exactly* one encrypted document is returned, which upon decryption reveals a single bit $b = \text{XOR}(x, y)$, (thereby allowing SEC to correctly compute the function XOR).

Composable Function Evaluation. While SEC harbors the capability of efficiently evaluating a binary Boolean gate, extending this to an n -size circuit is non-trivial. One way to use SEC in order to evaluate compositions of form $f_k(f_i(\cdot, \cdot), f_j(\cdot, \cdot))$ (where $f_i, f_j, f_k \in \{\text{XOR}, \text{AND}, \text{OR}\}$) is to first evaluate $f_i(\cdot, \cdot), f_j(\cdot, \cdot)$, send the result to the client who constructs the query for the *outer* function, and then execute the outer function query. However, this incurs extra communication rounds that scale linearly with the circuit depth.

To circumvent this issue, we augment SEC to perform query construction for the outer function on the (semi-honest) server itself. That is, instead of the need to decrypt the result (i.e. $\text{sval}_{\mathbf{w}_d^i, D_i}$ for $f_i(\cdot)$ and $\text{sval}_{\mathbf{w}_d^j, D_j}$ for $f_j(\cdot)$) of the inner function to recover D_i and D_j , we use mechanisms to directly map $\text{sval}_{\mathbf{w}_d^i, D_i}$ and $\text{sval}_{\mathbf{w}_d^j, D_j}$ to construct relevant tokens used by the outer function $f_k(\cdot)$. To do so, we extend Table 3 to allow compositions of XOR (as shown in Table 5). We note that compositions of AND/OR, as well as intermixing of XOR/AND/OR follow suit. For simplicity of exposition we exemplify using a XOR gate only.

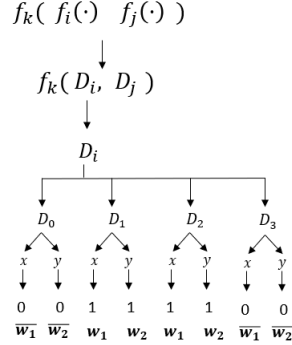


Figure 1: Construction of query for $f_k(\cdot)$ from the output of $f_i(\cdot)$ and $f_j(\cdot)$.

There are four possible outcomes for each of the inner XOR evaluation (denoted by plaintext document identifiers D_0, D_1, D_2, D_3), and each outcome can either behave as the first input (corresponding to bit x) or the second input (corresponding to bit y) for the outer function. For example, should the inner function output D_0 (corresponding to the evaluation of $\text{XOR}(0, 0)$; see Table 3), then for the outer function, either $x = 0$ or $y = 0$ depending on specific wiring topology of these gates (See Figure 1).

Therefore, according to Table 5, either keyword $\bar{\mathbf{w}}_1$ or $\bar{\mathbf{w}}_2$ shall be involved in the outer function's query. This mapping hence allows query construction of the outer function evaluation on the server side itself. For further compositions, (like the output f_k being used in some outer computation), the same table is *reused*; this prevents any exponential blowup in storage. Concretely, a single mapping form Table 5 is sufficient for evaluating composed functions of arbitrary depth. Table 5 appended with "special" terms and dummy documents entail the final search index for the XOR function. Similar construction follow for the search index of AND/OR gates as well.

Illustrative Example 1 (Composability). Consider a client who wishes to evaluate the circuit in Figure 2 using SEC without involving communication rounds with the client after inner function evaluations and using the *same* lookup table (Table 5; that is encrypted and offloaded to the server once at the beginning). The encrypted lookup tables for all three functions $\mathbf{f} = \{\text{XOR}, \text{AND}, \text{OR}\}$ are generated once and offloaded to the server. We emphasize

Table 5: Mapping between the result of an inner function (say $f_i(\cdot)$ here), to keywords (input bits) used in querying the outer function ($f_k(\cdot)$). Mapping of `doc_id` is directly extrapolated from Table 4. For ease of exposition, we omit “special” terms and dummy documents from this illustration; they are added to the search index similarly as explained previously.

Output of $f_i(\cdot)$	Mapped Keyword	doc_id
D_0	$\bar{\mathbf{w}}_1$	D_0, D_1
D_1	$\mathbf{w}_1$	D_2, D_3
D_2	$\mathbf{w}_1$	D_2, D_3
D_3	$\bar{\mathbf{w}}_1$	D_0, D_1
D_0	$\bar{\mathbf{w}}_2$	D_0, D_2
D_1	$\mathbf{w}_2$	D_1, D_3
D_2	$\mathbf{w}_2$	D_1, D_3
D_3	$\bar{\mathbf{w}}_2$	D_0, D_2

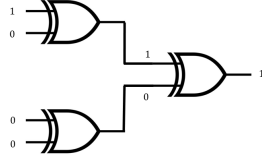


Figure 2: Evaluate: $f_{\text{XOR}}(f_{\text{XOR}}(1, 0), f_{\text{XOR}}(0, 0))$

here that the size of the lookup table is agnostic of the circuit being evaluated. For ensuring a single communication round between the client and server, the query construction for outer XOR needs to happen solely on the server side. For now, we assume $\text{ConstructQuery}(\mathbf{w}_d^k, \text{sval}_{\mathbf{w}_d^l, D_j}, b)$ (see Algorithm 8 for details) to abstract the following mapping: Given that $\text{sval}_{\mathbf{w}_d^l, D_j}$ is output by the inner function evaluation, ConstructQuery returns the search tokens of the form $\text{Token}_{\mathbf{w}_d^k, \mathbf{w}_i, D_j}$ depending on the mapping in Table 5. Bit b denotes whether $\text{sval}_{\mathbf{w}_d^l, D_j}$ is first or second input to the outer function. We defer details of the exact operation of ConstructQuery to Section 3.2. We also note that since *three* evaluations of XOR occur, we use a *distinct* “special” term for each. We now explain how the evaluation proceeds.

SEC first constructs a query $q_1 = (\mathbf{w}_d^1 \wedge \mathbf{w}_1 \wedge \bar{\mathbf{w}}_2)$ for $f_{\text{XOR}}(1, 0)$, resulting in $\text{sval}_{\mathbf{w}_d^1, D_2}$ ($\mathbf{w}_d^1$ is a randomly sampled “special” term); D_2 occurs in both $\mathbf{w}_1 \wedge \bar{\mathbf{w}}_2$, see Table 5). Likewise, SEC constructs a query $q_2 = (\mathbf{w}_d^2 \wedge \bar{\mathbf{w}}_1 \wedge \bar{\mathbf{w}}_2)$ for $f_{\text{XOR}}(0, 0)$, resulting in $\text{sval}_{\mathbf{w}_d^2, D_0}$ ($\mathbf{w}_d^2$ is a randomly sampled “special” term); D_0 occurs in both $\bar{\mathbf{w}}_1 \wedge \bar{\mathbf{w}}_2$, see Table 5). Note that the result of $f_{\text{XOR}}(1, 0)$ drives the first input of outer XOR. Thus, SEC invokes $\text{ConstructQuery}(\mathbf{w}_d^3, \text{sval}_{\mathbf{w}_d^1, D_2}, 1)$ to obtain $\text{Token}_{\mathbf{w}_d^3, \mathbf{w}_1, D_2}$ and $\text{Token}_{\mathbf{w}_d^3, \mathbf{w}_1, D_3}$ ($\mathbf{w}_d^3$ is a randomly sampled “special” term); $\mathbf{w}_1$ is chosen as the keyword for the first input of outer XOR because the output of $f_{\text{XOR}}(1, 0)$ gives D_2 , and from the corresponding row entry 3 in Table 5 we get $\mathbf{w}_1$; (D_2, D_3) are documents corresponding to $\mathbf{w}_1$; 1 in $\text{ConstructQuery}()$ denotes this is the first input of the gate). Likewise, since $f_{\text{XOR}}(0, 0)$ drives the second input of outer XOR, SEC invokes $\text{ConstructQuery}(\mathbf{w}_d^3, \text{sval}_{\mathbf{w}_d^2, D_0}, 0)$ to obtain $\text{Token}_{\mathbf{w}_d^3, \bar{\mathbf{w}}_2, D_0}$ and $\text{Token}_{\mathbf{w}_d^3, \bar{\mathbf{w}}_2, D_2}$ ($\mathbf{w}_d^3$ is a randomly sampled “special” term); $\bar{\mathbf{w}}_2$ is chosen as the keyword for the second input of outer XOR because the output of $f_{\text{XOR}}(0, 0)$ gives D_0 , and from the corresponding row entry 5 in Table 5 we get $\bar{\mathbf{w}}_2$; (D_0, D_2) are documents corresponding to $\bar{\mathbf{w}}_2$; 0 in $\text{ConstructQuery}()$ denotes this is the first second of the gate). Overall, for the outer XOR, the constructed query is $q_3 = (\mathbf{w}_d^3 \wedge \mathbf{w}_1 \wedge \bar{\mathbf{w}}_2)$. It is straightforward to see that the result of the functional composition, by design of Table 5 shall be $\text{sval}_{\mathbf{w}_d^3, D_2}$ (D_2 is

the common document between all four **Tokens** generated by **ConstructQuery()**. The server sends this *final* result to the client. Upon decryption, the client obtains the output plaintext bit 1 ($D_2 = \text{Enc}_k(1)$), which is the correct evaluation of the circuit. We emphasize that since the “special” term changes across all gate evaluations (due to the random selection of “special” terms for each gate by the client), from the server’s perspective, it can not correlate the underlying computations.

Reusability of Lookup Tables for Multiple Evaluations. SEC enables reusability of the same lookup table for multiple evaluations of the same/different circuits while ensuring data privacy guarantees. The client chooses a “special” term for evaluating a particular gate, hence for evaluating the same/different gate more than once, the client can randomly choose a unique “special” term for each gate evaluation. Since only the “special” term in a given conjunctive query is used to index into the encrypted look-up table in the memory and retrieve respective encrypted documents, choosing random “special” terms for each query (equivalent to each gate evaluation at same/different depth of the circuit) obfuscates the *access pattern* of the memory location accessed by the “special” terms. Thus even if the client computes the same function, say $\text{XOR}(x, y)$, twice on the same encrypted input, the server (adversary) cannot infer any correlation between the two computations. This ensures the reusability of the same encrypted look-up table for multiple circuit evaluations without any significant information leakage to the semi-honest server. This is a straightforward improvement from GCs that incorporate non-reusable lookup tables to perform encrypted circuit evaluation. The reusability of encrypted tables in SEC amortizes client’s communication overhead, since it sets the “special” terms linear in the circuit-size once and reuses them for multiple evaluations while preventing exponential growth in storage/communication. The client does not require prior knowledge of circuit composition. Concretely, for a publicly-known circuit with n gates, the search-index has k “special” terms ($k \geq n$). Reusability is then derived from client-controlled “permutation” of assigning unique “special” term to isomorphic gates across multiple runs, ensuring unique access-pattern across multiple gate evaluations. Concretely, the upper bound on reusability is the number of possible iterative derangements from a set of size k ($k \geq n$), i.e., $O(k)$. We remark that for practical circuits, a reasonable bound on k ensures sufficiently large re-executions with reasonable storage (see Table 2 and Section 8 for an example). Additionally, “special” terms are permuted using a Pseudo-Random Permutation primitive, for which the client maintains $O(1)$ state. More details follow in Section 6.

Illustrative Example 2 (Reusability). We demonstrate another scenario in this example, where the client wishes to consecutively evaluate the circuit in Figure 2 twice. Assume that for Evaluation 1, SEC constructs a query similar to the previous example. For Evaluation 2, the client assigns a different set of “special” terms and SEC constructs corresponding query as - $q_1 = (\mathbf{w}_d^3 \wedge \mathbf{w}_1 \wedge \bar{\mathbf{w}}_2)$ for $f_{\text{XOR}}(1, 0)$, resulting in $\mathbf{sval}_{\mathbf{w}_d^3, D_2}$. Likewise, a query $q_2 = (\mathbf{w}_d^1 \wedge \bar{\mathbf{w}}_1 \wedge \bar{\mathbf{w}}_2)$ for $f_{\text{XOR}}(0, 0)$, resulting in $\mathbf{sval}_{\mathbf{w}_d^1, D_0}$. As the result of $f_{\text{XOR}}(1, 0)$ drives the first input of outer XOR, SEC invokes **ConstructQuery**($\mathbf{w}_d^2, \mathbf{sval}_{\mathbf{w}_d^3, D_1}, 1$) to obtain **Token** $_{\mathbf{w}_d^2, \mathbf{w}_1, D_2}$ and **Token** $_{\mathbf{w}_d^2, \mathbf{w}_1, D_3}$ (corresponding to row entry 3 in Table 5). Likewise, since $f_{\text{XOR}}(0, 0)$ drives the second input of outer XOR, SEC invokes **ConstructQuery**($\mathbf{w}_d^2, \mathbf{sval}_{\mathbf{w}_d^1, D_0}, 0$) to obtain **Token** $_{\mathbf{w}_d^2, \bar{\mathbf{w}}_2, D_0}$ and **Token** $_{\mathbf{w}_d^2, \bar{\mathbf{w}}_2, D_2}$ (corresponding to row 5 in Table 5). Overall, for the outer XOR, the constructed query is $q_3 = (\mathbf{w}_d^2 \wedge \mathbf{w}_1 \wedge \bar{\mathbf{w}}_2)$ and the final result is $\mathbf{sval}_{\mathbf{w}_d^2, D_2}$. Note that the “special” terms used for q_1, q_2, q_3 are different hence, different **sval** and **Tokens** are generated for the same circuit in Evaluation 2. This obfuscates memory access pattern. However the result of the final evaluation will be the same (this ensures functional correctness).

It is to be noted that Table 5 is an abstract overview of the lookup table representation, hence for ease of explanation, we show that the entries related to a search tokens ($\{\mathbf{Token}_{\mathbf{w}_d^3, \bar{\mathbf{w}}_2, D_0}$

and $\text{Token}_{\mathbf{w}_d^3, \bar{\mathbf{w}}_2, D_2}$ for Evaluation 1 and $\{\text{Token}_{\mathbf{w}_d^2, \bar{\mathbf{w}}_2, D_0}$ and $\text{Token}_{\mathbf{w}_d^2, \bar{\mathbf{w}}_2, D_2}\}$ for Evaluation 2), correspond to the same row of the lookup table (row 5 in Table 5) for both evaluations. In practice, however, every entry corresponding to a “special” term is stored separately, hence although the content of the encrypted document is the same, the locations in memory will vary. This is crucial to prevent the server from correlating between two (similar/different) circuit evaluations, thereby rendering the reusability of the same lookup table for multiple evaluations secure, in terms of data privacy.

From the server’s perspective (in Figure 2), for gate 1, the server observes access patterns related to $\{\text{sval}_{\mathbf{w}_d^1, D_2}\}$ in Evaluation 1 and $\{\text{sval}_{\mathbf{w}_d^3, D_2}\}$ in Evaluation 2. Similarly, for gate 2, access patterns observed are $\{\text{sval}_{\mathbf{w}_d^2, D_0}\}$ in Evaluation 1 and $\{\text{sval}_{\mathbf{w}_d^1, D_0}\}$ in Evaluation 2. Likewise, for gate 3, the server observes access patterns for $\{\text{Token}_{\mathbf{w}_d^3, \mathbf{w}_1, D_2}, \text{Token}_{\mathbf{w}_d^3, \mathbf{w}_1, D_3}, \text{Token}_{\mathbf{w}_d^3, \bar{\mathbf{w}}_2, D_0}, \text{Token}_{\mathbf{w}_d^3, \bar{\mathbf{w}}_2, D_2}, \text{sval}_{\mathbf{w}_d^3, D_2}\}$ in Evaluation 1 and $\{\text{Token}_{\mathbf{w}_d^2, \mathbf{w}_1, D_2}, \text{Token}_{\mathbf{w}_d^2, \mathbf{w}_1, D_3}, \text{Token}_{\mathbf{w}_d^2, \bar{\mathbf{w}}_2, D_0}, \text{Token}_{\mathbf{w}_d^2, \bar{\mathbf{w}}_2, D_2}, \text{sval}_{\mathbf{w}_d^2, D_2}\}$ in Evaluation 2. Briefly, through client-controlled permutation, assignment of “special” terms to *non-isomorphic* gates¹³ prevents the server from correlating between two (similar/different) circuit evaluations, thereby making the reusability of the same lookup table for multiple evaluations secure, in terms of data privacy.

*Note. It is important to note that, for ease of exposition, we exemplified the working mechanism of SEC for solving a circuit with XOR gates only. An exactly similar execution methodology is used for evaluating any arbitrary circuit f with any combination of gates, $f \in \{\text{XOR}, \text{AND}, \text{OR}\}$, since both *sval* and *Token* are function agnostic.*

Security and Communication Complexity. We stress that the same look-up tables can be used for re-evaluations of same circuit through client-controlled *permutation* of the pool of “special” terms available in SEC’s encrypted look-up tables. This ensures no two gates in the *same* circuit share a common “special” term, thereby preventing the server from inferring any correlation between two computations. We elaborately discuss and analyze the leakage profile of SEC in Section 6. The bulk of communication overhead is a *one-time* setup¹⁴ where the client generates and offloads data structures related to SEC to the server. For a circuit consisting of n gates, we assign a unique “special” term sequence to each circuit for each re-evaluation, and some *dummy* documents (to maintain a uniform frequency of all keywords in the encrypted database). The space complexity¹⁵ of SEC thus scales linearly with $\mathcal{O}(k)$ for client controlled k “special” terms ($k \geq n$). Higher k implies more evaluations, with added server storage cost (that is acceptable; see concrete storage in Table 2 and Section 8).

1.3 Related Work

Garbled Circuits. It was first introduced by Yao in his groundbreaking work on secure function evaluation [51]. Due to their versatility, various garbling techniques (for arithmetic/Boolean circuits) [5, 34, 41, 51] have been developed over the years. GCs typically provide privacy for the input/output bits and entire circuit that is being computed. However, such constructions have an inherent disadvantage of not being reusable that makes it unnameable for practical deployment. As such, there have been attempts to construct reusable Garbled Circuits [34],

¹³If in Evaluation 2, the client had assigned $\mathbf{w}_d^2$ to gate 2, then this assignment is isomorphic to that in Evaluation 1. Consequently, the server’s leakage profile for gate 2 is $\{\text{sval}_{\mathbf{w}_d^2, D_0}\}$ for both evaluations, hence leaking that both evaluations have the *same type of gate*. Note that, however, neither the gate description (i.e. whether it is AND/OR/XOR) nor the exact plaintext bit in $\{\text{sval}_{\mathbf{w}_d^2, D_0}\}$ is leaked, since this information is encrypted, thereby still protecting data privacy.

¹⁴Amortized by enabling multiple re-evaluations.

¹⁵Keywords specific to functional evaluation are $\mathcal{O}(1)$; refer Table 4 and Table 5.

but the underlying primitive used is Functional Encryption (using FHE as a black box) which is computationally expensive.

Fully Homomorphic Encryption (FHE). In practice, computing over encrypted outsourced databases has used sophisticated and highly structured primitives, such as Fully Homomorphic Encryption (FHE) [2, 3, 20, 27, 29, 30, 31, 33] which views generic computations as either arithmetic or Boolean circuits (hence are typically expensive), and provides an all-or-nothing flavor of security. FHE has recently gained traction in the cryptographic literature for privacy-preserving computation with rich functionalities along with the *ideal* notion of privacy. However, as each primitive function evaluation is realized by explicit circuit evaluation followed by an expensive *bootstrapping* operation. FHE has prohibitively high computation costs and storage overheads.

Searchable Symmetric Encryption (SSE). SSE schemes [15, 18, 24, 47] provision users with *search* capabilities over symmetrically encrypted data. There exist today efficient SSE schemes that support conjunctive (and more general Boolean) queries [14, 15, 45]. An *ideal* SSE construction using Oblivious RAM (ORAM) promises oblivious memory access patterns (and hence *no* leakage) but is hard to actualize in hardware, is closed-source, and has not been tested against scaled databases [17]. Modern SSE schemes thus trade-off security for efficiency. These schemes allow the server to learn “some” information during query execution, detailed by their *leakage* profile. While SSE schemes are extremely fast and highly scalable with arbitrarily large real-world datasets, their restricted functionality (to only *search*) renders them inapt for practical deployment in an encrypted computation framework. In this work, we leverage the efficient look-up capabilities of SSE while extending the limited functionality of existing SSE schemes to supporting encrypted computations of arbitrary Boolean functions.

1.4 How SEC differs from GC and HE

In its essence, SEC guarantees privacy of input and output bits to and from a Boolean function. The rationale behind the construction of SEC is to provide an efficient and accelerated alternative to the existing state-of-the-art general purpose encrypted computing frameworks while leaking some non-trivial information that is proven to be benign. We would like to emphasize that the security guarantee of SEC is based on the IND-CPA security of a symmetric key encryption scheme. It is important to mention that the function evaluated by SEC is assumed to publicly known. We provide an elaborate comparison of SEC wrt. the most closely related encrypted computation frameworks in literature.

Garbled Circuits. We emphasize that with a single setup of $O(n)$ special-terms and its client-controlled permutation, SEC ensures multiple executions of the same circuit without leaking any equality-correlation to the server. This is the core difference between SEC and GC. While GC uses two inputs to evaluate a single binary-gate by indexing into its translation tables, SEC uses three inputs¹⁶ to evaluate the same gate. The extra input neither encodes the actual bit nor participates in the computation, but just randomizes memory accesses for the same gate across multiple executions. This allows up to $O(n!)$ re-executions of the same circuit with a single setup, while GC can tolerate just one execution. “**Illustrative Example 2**” demonstrates how SEC can execute the same circuit (with same inputs) twice without leaking correlations (unlike GC with a single setup). Footnote 8 gives an example where an *incorrect* permutation of “special” terms leaks to leakage in SEC. It is the client’s responsibility to ensure the same “special” terms are not assigned to isomorphic gates.

SEC also differs from GCs that use structured encryption underneath. For instance, Kamara et. al. proposed a garbled circuit construction from a structured encryption scheme in [41], which

¹⁶Two inputs corresponding to gate input wires, and an additional “special” term.

seems closely related to SEC in terms of its design choice and functionality. We emphasize, however, SEC is inherently distinct from the construction proposed in [41], as it relies on the efficient encrypted look-up operation of a conjunctive SSE scheme to evaluate arbitrary Boolean functions. Essentially [41] reduces the problem of designing special-purpose garbled circuits to the problem of designing structured encryption schemes. Whereas, SEC reduces the problem of designing an arbitrary Boolean function evaluation framework on encrypted input bits to the problem of designing a conjunctive SSE scheme. As such SEC ensures data-privacy guarantees, while assuming the circuit is publicly known. SEC entails a single round of setup for $O(n!)$ circuit evaluations (n -size circuit), without leaking any correlation between the circuit evaluations¹⁷. On the contrary [41] requires the setup phase to run for every evaluation, i.e. for an n -sized circuit, a garbled circuit of size n needs to be setup every-time for evaluating the circuit without leaking any information.

Fully Homomorphic Encryption. In this work, we provide an encrypted computation framework that is more efficient than FHE in terms of computation and storage complexity, while providing similar security guarantees of data-privacy. The construction of SEC fundamentally differs from FHE schemes because SEC bypasses explicit circuit evaluation, hence the consequent need for bootstrapping operation, that essentially contributes to FHE’s computational bottleneck.

We use TFHE as the concrete implementation of FHE for comparison with SEC. Functional bootstrapping in TFHE enables the evaluation of homomorphic functions, which can be used to perform encrypted table lookups. The lookup is performed using a homomorphic truth table representation and lookup table evaluation. It enables arbitrary function computation over encrypted data. Lookups are performed using blind rotation and GGSW ciphertext [33] manipulations. The process is computationally expensive because bootstrapping is costly. The fundamental difference of TFHE with SEC is that the later relies on the efficient encrypted lookup operations of a conjunctive SSE scheme. The lookup operation in SSE is simpler and extremely efficient. On the other hand, while TFHE can support arbitrary number of evaluations with a single setup, SEC can manage bounded evaluations. We argue however, that such a bound is asymptotically large for practical deployments: for a single setup of a circuit of size n , SEC bounds the number of re-evaluations it can perform by $O(n!)$.

With respect of comparison of security, SEC provides data-privacy guarantee by encrypting the input/output bits to/from a gate, using an IND-CPA secure symmetric-key encryption scheme. The incorporation of an SSE scheme as a black-box makes it straightforward to see that the leakages that exist in state-of-the-art SSE schemes will affect the overall security of SEC. However we prove that the leakages incurred by the underlying SSE does not transitively reflect in the leakage profile of SEC due to appropriate measures and design choices made. We claim that the leakages from the underlying SSE are handled in SEC, which renders them inconsequential and preserves its data-privacy guarantees¹⁸. Hence, although unlike FHE, SEC is prone to underlying SSE leakages, we underscore that such leakages are benign and has no adverse affect on the overall security of the scheme. We provide elaborate details on the security analysis of SEC in Section 6.

Leveled Homomorphic Encryption. As seen from Table 1, Leveled Homomorphic Encryption (LHE) [11, 13, 35] allows evaluating circuits of bounded size without requiring bootstrap-

¹⁷Note, the privacy of input bits to the circuit and output bit after evaluation is guaranteed by the IND-CPA symmetric-key encryption scheme with which the input and output bits are encrypted

¹⁸Informally, SEC uses three inputs to evaluate a binary gate, and ties SSE related leakages to the “special” term, which does not encode the real gate inputs. In other words, although SEC leaks, its leakages are tied to the “special” terms and not the real gate inputs. By randomizing the permutation of the “special” terms across different evaluations, SEC prevents any exploitable leakage.

ping, and provides a balance between efficiency and expressiveness. However, LHE can only evaluate circuits up to a fixed depth (e.g., $10 \sim 20$ [37], $10 \sim 50$ [39] multiplicative levels), causing a decryption failure due to excessive noise if computations exceed this depth. Whereas SEC supports $O(n!)$ evaluations of a circuit of size n , where n is a client-controlled parameter and not specific to the design of SEC. This is an important feature as it accelerates the scalability of SEC and renders it suitable for practical real-world deployment.

2 Preliminaries and Background

We present preliminary concepts and background in this section. Table 6 lists basic notations used in this paper. Any other notation used is defined in-place within the context of the main text.

Table 6: Summary of notations

λ	security parameter
id/doc_id	document identifier
$\mathbf{w}$	a keyword
$\mathcal{W}$	dictionary of keywords $\mathcal{W} = \{\mathbf{w}_1, \dots, \mathbf{w}_N\}$
$\mathbf{DB}$	database $(\text{id}_j, \mathbf{w}_i)_{i=1}^N \in \mathbf{DB}$
$\mathbf{DB}(\mathbf{w})$	all documents containing $\mathbf{w}$
n	max. number of keywords per conjunctive query.
$x \xleftarrow{\$} \chi$	uniformly sampling x from χ
$x = \mathcal{A}$	x is output of a deterministic algorithm
$x \leftarrow \mathcal{A}'$	x is output of a randomized algorithm

2.1 Conjunctive SSE: Syntax and Security Model

A *Conjunctive Searchable Symmetric Encryption* scheme (CSSE) provisions the client with *conjunctive* search capability (i.e. search Boolean queries of the form $\mathbf{w}_1 \wedge \mathbf{w}_2 \wedge \dots \wedge \mathbf{w}_n$) over an encrypted database. A CSSE scheme can be formally defined as an ensemble of four polynomial-time algorithms $\{\text{KEYGEN}, \text{ENCRYPT}, \text{GENTOKEN}, \text{SEARCH}\}$ ¹⁹ such that:

- $\text{KEYGEN}(\lambda)$ is a probabilistic algorithm that takes the security parameter λ as input. The output of this algorithm is the client’s secret key sk .
- $\text{ENCRYPT}(\text{sk}, \mathbf{DB})$ is a probabilistic algorithm that takes as input the client secret key sk and a plain database $\mathbf{DB}$. The output is an encrypted database $\mathbf{EDB}$.
- $\text{GENTOKEN}(\text{sk}, q = \mathbf{w}_1 \wedge \dots \wedge \mathbf{w}_n)$ is a deterministic algorithm executed by the client that takes as input secret key sk and a conjunctive query $q = \mathbf{w}_1 \wedge \dots \wedge \mathbf{w}_n$. It generates *search tokens* (st_q) corresponding to the conjunctive query q as the output.
- $\text{SEARCH}(\mathbf{EDB}, \text{st}_q)$ is a deterministic algorithm executed by the *server* that takes as input $\mathbf{EDB}$ and the search token st_q corresponding to a conjunctive query q . It returns the encrypted document identifiers $\mathbf{DB}(\text{st}_q)$ corresponding to the conjunction $q = \mathbf{w}_1 \wedge \dots \wedge \mathbf{w}_n$ as output.

Correctness. A CSSE scheme is said to be correct if for an $\mathbf{EDB}$ generated from a $\mathbf{DB}$ using ENCRYPT , for a search token st_q generated by GENTOKEN from any conjunctive Boolean query

¹⁹Our syntax for a conjunctive SSE is different from the syntax of traditional conjunctive SSE scheme $\{\text{SETUP}, \text{GENTOKEN}, \text{SEARCH}\}$, but the underlying functionality is exactly similar

q formed over the keywords $\mathbf{w}_i$ in $\mathcal{W}$, the SEARCH routine returns a set of ids as result which is the same as $\mathbf{DB}(q)$ with high probability.

Security. The security of a CSSE scheme is parameterized by a *leakage function* $\mathcal{L}$, which encapsulates the information that can be learnt (potentially by an adversary) from the encrypted database and query transcripts. Formally, the security notion says that the server's view during an adaptive attack (where the server selects the database and queries) can be simulated given only the output of $\mathcal{L}$.

Let $\text{CSSE} = \{\text{KEYGEN}, \text{ENCRYPT}, \text{GENTOKEN}, \text{SEARCH}\}$ be a CSSE scheme, and let $\mathcal{L}$ be a stateful algorithm. For algorithms $\mathcal{A}$ (denoting the adversary) and SIM (denoting a simulator), we define the experiments (algorithms) $\mathbf{Real}_{\mathcal{A}}^{\text{CSSE}}(\lambda)$ and $\mathbf{Ideal}_{\mathcal{A}, \text{SIM}}^{\text{CSSE}}(\lambda)$, as in Algorithm 1 and Algorithm 2, respectively (see Section 2.2). We say that CSSE is $\mathcal{L}$ -semantically-secure against adaptive attacks if for all adversaries $\mathcal{A}$ there exists an algorithm SIM such that

$$|\Pr[\mathbf{Real}_{\mathcal{A}}^{\text{CSSE}}(\lambda) = 1] - \Pr[\mathbf{Ideal}_{\mathcal{A}, \text{SIM}}^{\text{CSSE}}(\lambda) = 1]| \leq \text{negl}(\lambda).$$

In these experiments, the leakage function for CSSE is expressed as

$$\mathcal{L}_{\text{CSSE}} = (\mathcal{L}_{\text{CSSE}}^{\text{ENCRYPT}}, \mathcal{L}_{\text{CSSE}}^{\text{GENTOKEN}}, \mathcal{L}_{\text{CSSE}}^{\text{SEARCH}}),$$

where $\mathcal{L}_{\text{CSSE}}^{\text{ENCRYPT}}$ encapsulates the leakage to an adversarial server during the ENCRYPT phase, $\mathcal{L}_{\text{CSSE}}^{\text{GENTOKEN}}$ encapsulates the leakage to an adversarial server during the GENTOKEN phase, and $\mathcal{L}_{\text{CSSE}}^{\text{SEARCH}}$ encapsulates the leakage to an adversarial server during each execution of the SEARCH protocol.

2.2 Adaptive Security of CSSE

We present the **Real** and **Ideal** experiments for the security analysis of a conjunctive SSE scheme CSSE in this section. In these experiments, the leakage function for CSSE is expressed as

$$\mathcal{L}_{\text{CSSE}} = (\mathcal{L}_{\text{CSSE}}^{\text{ENCRYPT}}, \mathcal{L}_{\text{CSSE}}^{\text{GENTOKEN}}, \mathcal{L}_{\text{CSSE}}^{\text{SEARCH}}),$$

where $\mathcal{L}_{\text{CSSE}}^{\text{ENCRYPT}}$ encapsulates the leakage to an adversarial server during the ENCRYPT phase, $\mathcal{L}_{\text{CSSE}}^{\text{GENTOKEN}}$ encapsulates the leakage to an adversarial server during the GENTOKEN phase, and $\mathcal{L}_{\text{CSSE}}^{\text{SEARCH}}$ encapsulates the leakage to an adversarial server during each execution of the SEARCH protocol.

Algorithm 1 Experiment $\mathbf{Real}_{\mathcal{A}}^{\text{CSSE}}(\lambda)$

```

1: function  $\mathbf{Real}_{\mathcal{A}}^{\text{CSSE}}(\lambda)$ 
2:    $N \leftarrow \mathcal{A}(\lambda)$ 
3:    $(\mathbf{sk}, \mathbf{s}_0, \mathbf{EDB}_0) \leftarrow \text{CSSE.ENCRYPT}(\lambda, N)$ 
4:   for  $k \leftarrow 1$  to  $Q$  do
5:     Let  $q_k \leftarrow \mathcal{A}(\lambda, \mathbf{EDB}_{k-1}, \tau_1, \dots, \tau_{k-1})$ 
6:     Let  $(\mathbf{s}_k, \mathbf{EDB}_k, \mathbf{DB}(q_k)) \leftarrow$ 
        $\text{CSSE.SEARCH}(\mathbf{sk}, \mathbf{s}_{k-1}, q_k; \mathbf{EDB}_{k-1})$ 
7:     Let  $\tau_k$  denote the view of the adversary after
       the  $k^{\text{th}}$  query
8:    $b \leftarrow \mathcal{A}(\lambda, \mathbf{EDB}_Q, \tau_1, \dots, \tau_Q)$ 
9:   return  $b$ 
```

Algorithm 2 Experiment $\text{Ideal}_{\mathcal{A}, \text{SIM}}^{\text{CSSE}}(\lambda, Q, \mathcal{L})$

```

1: function  $\text{Ideal}_{\mathcal{A}, \text{SIM}}^{\text{CSSE}}(\lambda, Q, \mathcal{L})$ 
2:   Parse the leakage function  $\mathcal{L}$  as:
      $\mathcal{L} = (\mathcal{L}_{\text{CSSE}}^{\text{ENCRYPT}}, \mathcal{L}_{\text{CSSE}}^{\text{SEARCH}})$ .
3:    $(\text{SSIM}, \mathbf{EDB}_0) \leftarrow \text{SIM}_{\text{SETUP}}(\mathcal{L}^{\text{ENCRYPT}}(\lambda, N))$ 
4:   for  $k \leftarrow 1$  to  $Q$  do
5:     Let  $q_k \leftarrow \mathcal{A}(\lambda, \mathbf{EDB}_{k-1}, \tau_1, \dots, \tau_{k-1})$ 
6:     Let  $(\text{SSIM}, \mathbf{EDB}_k, \tau_k) \leftarrow \text{SIM}_{\text{SEARCH}}$ 
        $(\text{SSIM}, \mathcal{L}_{\text{CSSE}}^{\text{SEARCH}}(q_k); \mathbf{EDB}_{k-1})$ 
7:     Let  $\tau_k$  denote the view of the adversary after
       the  $k^{\text{th}}$  query
8:    $b \leftarrow \mathcal{A}(\lambda, \mathbf{EDB}_Q, \tau_1, \dots, \tau_Q)$ 
9:   return  $b$ 

```

2.3 Oblivious Cross-Tag Protocol (OXT): Overview

We provide a brief understanding of the technical details of the OXT protocol. OXT [16] relies on specially structured pseudo-random functions that can be incorporated using discrete-log hard groups. The idea is that the client encrypts the data (using symmetric-key encryption) and offloads it to the server (*honest-but-curious*). The client queries the server with a conjunction of keywords to which it returns a set of encrypted pointers that point to documents containing all the client's queried keywords. The client decrypts these pointers locally to obtain the required documents matching the conjunctive query. The server does not have the ability to perform decryption nor can it learn any information about the queried keywords. The entire protocol constitutes one-round of interaction/communication between the client and the server. This is delegated by an *oblivious shared computation of cross-tags* between client and server. In order to achieve minimum communication complexity while ensuring privacy of the client's queried keywords and corresponding documents, OXT incorporates an *oblivious cross-tag* generation process. The cross-tag is computed using blinded exponentiation in prime order cyclic groups, analogous to *Diffie-Hellman based oblivious PRF*. It pre-computes the blinding part of the oblivious computation and stores them in encrypted form at the server. During search, the client sends tokens to the server using which it unlocks these pre-computed values and computes the cross-tag *obliviously* using which it matches the documents corresponding to the queried keywords.

The core technical idea of OXT is the process of oblivious cross-tag generation, that relies on public-key computations in a discrete-log hard group. The hardness assumption is based on the fact that *DDH assumption* is conjectured to hold in a prime order subgroup of Z_p^* (p is a large prime). The inherent reliance of OXT on discrete-log hard groups renders it vulnerable to quantum attacks and can be effectively broken by a scalable quantum computer. Therefore, although OXT provides security against efficient adaptive adversaries in classical setting, it is not secure in the post-quantum setting. The motivation of our work is hence, to develop a post-quantum secure construction of OXT that preserves its asymptotic search and communication complexity while ensuring its scalability with arbitrary large datasets.

The Cross-Tag. The most fundamental component of OXT which is incorporated to check for the presence of a keyword in a particular document without leaking any extra information to the server, is the *cross-tag* (denoted as xtag in the paper). The fundamental building block that makes OXT one of the most efficient, highly scalable and secure conjunctive SSE scheme is an *oblivious computation of the xtag at the server*, which is incorporated using a DH-based

oblivious PRF type computation. The idea is, for every keyword and every document in which it is present, the client pre-computes a **xtag** (an element in the prime-order subgroup of Z_p^*) and stores it in a data-structure called **XSet** which is offloaded to the server.

$$\mathbf{xtag} \leftarrow g^{F_p(K_X, w) \cdot F_p(K_I, \text{id})}$$

where, g is the generator of the subgroup of Z_p^* and F_p is a PRF that takes as input a keyword or a document identifier and outputs an element in Z_p^* .

Along with the queried keywords, the server receives some tokens (**xtoken**) from the client (also an element in the prime-order subgroup of Z_p^*) during any conjunctive search. The beauty of OXT lies in the fact that the whole process of **xtag** computation by the server takes place obliviously without revealing the keyword-document pair for which the **xtag** is being computed. This is done by raising the **xtoken** to a blinded value y (an element in Z_p^*) which is pre-computed by the client and stored at the server.

3 Symmetric Encrypted Computation

We introduce Symmetric Encrypted Computation (SEC) as an efficient framework for fast outsourced privacy-preserved Boolean circuit evaluation in the symmetric-key setting. SEC supports circuit evaluation of arbitrary size via encrypted look-up and a single round of communication between the client and the remote server. SEC guarantees reusability of the same look-up table for multiple evaluations without leaking any substantial non-trivial information to the server. We begin by presenting the syntax definition of SEC in this section before delving into the elaborate technical details of the framework subsequently.

3.1 Syntax of SEC

We briefly explain a general syntax of our proposed primitive here. It uses a static conjunctive SSE construction CSSE as a black-box (refer to Section 2.1 for details on syntax and security definitions). We assume a single (honest) client and a (semi-honest) server setting. SEC is abstracted as a tuple of four polynomial-time algorithms {KEYGEN, ENCRYPT, EVALUATE, DECRYPT}, as defined below:

- **KEYGEN**(λ): A probabilistic algorithm executed by the client that takes as input the security parameter λ . It outputs a client secret key **sk** and a public parameter **pp**.
- **ENCRYPT**(**ke**, $x_1, \dots, x_p$): A probabilistic algorithm executed by the client. It takes the client's secret key **ke** and a p -bit input $\{x_1, \dots, x_p\}$. The output is a ciphertext $c = \{c_1, \dots, c_p\}$.
- **EVALUATE**(f_{desc} , $c_1, \dots, c_p$, **pp**): A deterministic algorithm executed by the server that takes as input a description of a circuit f_{desc} that is to be evaluated, the encrypted input bits $c = \{c_1, \dots, c_p\}$ and the public parameter **pp**. The server returns the encrypted evaluation of the circuit **eval** to the client.
- **DECRYPT**(**ke**, **eval**): A deterministic algorithm executed by the client with its secret-key **ke** and an encrypted evaluation **eval** as input, which outputs the decrypted **result**.

Correctness. SEC is said to be functionally correct if for security parameter λ , and for the following sequence of operations:

$$\begin{aligned}
\text{sk}, \text{pp} &\leftarrow \text{SEC.KEYGEN}(\lambda) \\
\{c_1, \dots, c_p\} &\leftarrow \text{SEC.ENCRYPT}(\text{ke}, x_1, \dots, x_p) \\
\text{eval} &= \text{SEC.EVALUATE}(f_{\text{desc}}, c_1, \dots, c_p, \text{pp}) \\
\text{result} &= \text{SEC.DECRYPT}(\text{ke}, \text{eval}),
\end{aligned}$$

The following holds with certainty:

$$\Pr[\text{result} = f_{\text{desc}}(x_1, \dots, x_p)] = 1,$$

Security. SEC guarantees data privacy: privacy of inputs to a function and output of the evaluation by encrypting them using an IND-CPA secure symmetric-key encryption. Formally, SEC is said to be adaptively secure with respect to a leakage function $\mathcal{L}_{\text{SEC}} = \{\mathcal{L}_{\text{SEC}}^{\text{KEYGEN}}, \mathcal{L}_{\text{SEC}}^{\text{ENCRYPT}}, \mathcal{L}_{\text{SEC}}^{\text{EVALUATE}}\}$ if for any PPT adversary $\mathcal{A}$, for a p -bit input $x_1, \dots, x_p$, there exists a PPT simulator $\text{SIM} = \{\text{SIM}_{\text{KEYGEN}}, \text{SIM}_{\text{ENCRYPT}}, \text{SIM}_{\text{EVALUATE}}\}$ such that the following holds:

$$|\Pr[\text{Real}_{\mathcal{A}}^{\text{SEC}}(\lambda) = 1] - \Pr[\text{Ideal}_{\mathcal{A}, \text{SIM}}^{\text{SEC}}(\lambda, \mathcal{L}) = 1]| \leq \text{negl}(\lambda),$$

where the “real” experiment $\text{Real}_{\mathcal{A}}^{\text{SEC}}$ and the “ideal” experiment $\text{Ideal}_{\mathcal{A}}^{\text{SEC}}$ are as described in Algorithm 3 and Algorithm 4. $\mathcal{L}_{\text{SEC}}^{\text{KEYGEN}}$ captures the leakage from SEC.KEYGEN, $\mathcal{L}_{\text{SEC}}^{\text{ENCRYPT}}$ captures the leakage from SEC.ENCRYPT, and $\mathcal{L}_{\text{SEC}}^{\text{EVALUATE}}$ captures the leakage from SEC.EVALUATE.

Algorithm 3 Experiment $\text{Real}_{\mathcal{A}}^{\text{SEC}}(\lambda)$

```

1: function  $\text{Real}_{\mathcal{A}}^{\text{SEC}}(\lambda)$ 
2:    $(\text{sk}, \text{pp}) \leftarrow \text{KEYGEN}(\lambda)$ 
3:   for  $k \leftarrow 1$  to  $y$   $\triangleright y = \text{poly}(\lambda)$ 
4:     do
5:       Let  $[x_1, \dots, x_p]_k \leftarrow \mathcal{A}(\lambda, \text{pp}, \tau_1, \dots, \tau_{k-1})$ 
6:       Let  $[c_1, \dots, c_p]_k \leftarrow \text{SEC.ENCRYPT}(\text{sk}, [x_1, \dots, x_p]_k)$ 
7:       Let  $f_{\text{desc}}([c_1, \dots, c_p]_k) \leftarrow \text{SEC.EVALUATE}(f_{\text{desc}},$ 
8:          $\text{pp}, [c_1, \dots, c_p]_k)$ 
9:        $(\tau_k \text{ denote } \mathcal{A}'\text{s view after the } k^{\text{th}} \text{ evaluation})$ 
10:   $b \leftarrow \mathcal{A}(\lambda, \text{pp}, f_{\text{desc}}, \tau_1, \dots, \tau_y)$ 
11:  return  $b$ 

```

3.2 SEC Construction

The fundamental goal of SEC is to compute arbitrary sized Boolean circuits using encrypted lookup tables while ensuring data-privacy, in a single round of communication between the client and server. The three universal Boolean function set supported by SEC are $f = \{f_{\text{XOR}}, f_{\text{AND}}, f_{\text{OR}}\}$. SEC incorporates a black-box conjunctive SSE construction to *compute* Boolean functions on encrypted inputs by performing searches over encrypted lookup tables. We refer the reader to Section 1.2 for an overview of SEC, and proceed with the construction here.

SEC.KeyGen. Algorithm 6 formally specifies the KEYGEN procedure. This routine is executed by the client, and it outputs a client secret key sk along with a public parameter pp . The public parameter pp consists of ① encrypted lookup tables (i.e., encrypted search indices) for $f_{\text{XOR}}, f_{\text{AND}}, f_{\text{OR}}$, and ② encrypted search tokens st (i.e., the token set). Once created, pp is outsourced to the server.

Algorithm 4 Experiment $\text{Ideal}_{\mathcal{A}, \text{SIM}}^{\text{SEC}}(\lambda, \mathcal{L})$

```

1: function  $\text{Ideal}_{\mathcal{A}, \text{SIM}}^{\text{SEC}}(\lambda, \mathcal{L})$ 
2:   Parse the leakage function  $\mathcal{L}$  as:
      $\mathcal{L} = (\mathcal{L}_{\text{SEC}}^{\text{KEYGEN}}, \mathcal{L}_{\text{SEC}}^{\text{ENCRYPT}}, \mathcal{L}_{\text{SEC}}^{\text{EVALUATE}})$ .
3:    $(\text{SSIM}, \text{pp}) \leftarrow \text{SIM}_{\text{KEYGEN}}(\mathcal{L}_{\text{SEC}}^{\text{KEYGEN}}(\lambda))$ 
4:   for  $k \leftarrow 1$  to  $y$   $\triangleright y = \text{poly}(\lambda)$ 
     do
5:     Let  $[x_1, \dots, x_p]_k \leftarrow \mathcal{A}(\lambda, \text{pp}, \tau_1, \dots, \tau_{k-1})$ 
6:     Let  $(\text{SSIM}, [c_1, \dots, c_p]_k) \leftarrow \text{SIM}_{\text{ENCRYPT}}(\text{SSIM},$ 
        $\mathcal{L}_{\text{SEC}}^{\text{ENCRYPT}}([x_1, \dots, x_p]_k); \text{pp})$ 
7:     Let  $f_{\text{desc}}([c_1, \dots, c_p]_k) \leftarrow \text{SIM}_{\text{EVALUATE}}(\text{SSIM},$ 
        $\mathcal{L}_{\text{SEC}}^{\text{EVALUATE}}(f_{\text{desc}}, \text{pp}, [c_1, \dots, c_p]_k))$ 
      $(\tau_k \text{ denote } \mathcal{A}'\text{s view after the } k^{\text{th}} \text{ evaluation})$ 
8:    $b \leftarrow \mathcal{A}(\lambda, \text{pp}, f_{\text{desc}}, \tau_1, \dots, \tau_y)$ 
9:   return  $b$ 

```

Algorithm 5 GENDB

Input: Security parameter λ

Output: DB

```

1: function GENDB( $\lambda$ )
2:    $f = \{f_{\text{XOR}}, f_{\text{AND}}, f_{\text{OR}}\}$ 
3:    $\text{Input\_set} = \{(x, y) \in \{0, 1\}^2\}$ 
4:   for  $i \in 1$  to  $|f|$  do
5:     for  $j \in 0$  to  $|\text{Input\_set}| - 1$  do
6:        $b = f_i(\text{Input\_set}[j])$ 
7:        $D_j \leftarrow \Pi_{\text{sym}}.\text{ENCRYPT}_{\text{ke}}(b)$ 
8:        $\text{DB}_{f_i} = \text{DB}_{f_i} \cup \{\text{MAP}(D_j, \text{Input\_set}[j])\}$ 
        $\triangleright \text{MAP is used as an abstraction of Table 4}$ 
9:    $\text{DB} = \text{DB} \cup \{\text{DB}_{f_i}\}$ 
10:  return DB

```

Concretely, first a helper function GENDB generates **DB** (see Algorithm 5), which essentially comprises the plaintext mappings between keywords and encrypted documents (Table 3 and Table 5). **DB** is then encrypted into SSE specific data structures using CSSE.ENCRYPT (See Section 2) to generate the encrypted search index or **EDB**²⁰, following the discussion established in Section 1.2. Thereafter, the specific search tokens **st** are generated by invoking CSSE.GENTOKEN (See Section 2). The size of **TokenSet** is therefore $\mathcal{O}(k)$, assuming a set of k ($k \geq n$) distinct “special” terms $\{\mathbf{w}_d^1, \mathbf{w}_d^2, \dots, \mathbf{w}_d^k\}$ for a circuit with n gates²¹. Recall from Section 1.2 that **ConstructQuery** routine (Algorithm 8) maps the output of one (inner) SEC evaluation to obtain search tokens for the *next* query (outer function evaluation). SEC thereby stores all pairwise mappings in $\{\mathbf{w}_d^1, \mathbf{w}_d^2, \dots, \mathbf{w}_d^k\}$ in the table²² to be used by the **ConstructQuery** routine. Concretely, *any* “special” term- document pair, $\mathbf{w}_d^k - D_j$ (i.e. $\text{sval}_{\mathbf{w}_d^k, D_j}$) can be used to obtain search tokens belonging to any other “special” term $\mathbf{w}_d^{k'}$: $\text{Token}_{\mathbf{w}_d^{k'}, \mathbf{w}_i, D_j}$ without re-

²⁰Refer to Appendeix 2.1 for conjunctive SSE syntax and security model.

²¹The analysis subsumes $\mathcal{O}(1)$ number of keywords for input combinations (i.e. $\mathbf{w}_1$ and $\mathbf{w}_2$), as well as $\mathcal{O}(1)$ number of documents ($D_j : j \in \{0, 1, 2, 3\}$)

²²Implemented either as a membership test (thereby allowing use of efficient Bloom Filters), or through a two-dimensional dictionary, as done in state-of-the-art SSE schemes.

quiring exponential storage. Finally, all the SEC specific data structures are offloaded to the server.

Note that KEYGEN is a *one-time* routine, executed only once at the beginning. We emphasize that by trading off storage, SEC achieves the capability of randomizing the order of “special” terms $\{\mathbf{w}_d^1, \mathbf{w}_d^2, \dots, \mathbf{w}_d^k\}$ across *different* executions of circuits, thereby achieving reusability of the same encrypted lookup tables for multiple evaluations. The client-controlled permutation of these “special” terms occurs during SEC.EVALUATE.

Algorithm 6 SEC.KEYGEN

Input: Security parameter λ

Output: Client’s secret key $\mathbf{sk}$ and a public parameter $\mathbf{pp}$

```

1: function SEC.KEYGEN( $\lambda$ )
2:   Samples a uniformly random key  $\mathbf{ke}$  for an IND-CPA Symmetric-key Encryption scheme:  $\Pi_{\text{sym}} =$ 
   ( $\text{KEYGEN}, \text{ENCRYPT}_{\mathbf{ke}}, \text{DECRYPT}_{\mathbf{ke}}$ )
3:    $\mathbf{DB} \leftarrow \text{GENDB}(\lambda)$ 
4:    $(\mathbf{EDB}, \mathbf{sk}) \leftarrow \text{CSSE.ENCRYPT}(\mathbf{DB}, \lambda)$ 
5:   for all conjunctive query  $q$  of keywords in  $\mathbf{DB}$  do
6:      $\{\mathbf{st}_q\} = \text{CSSE.GENTOKEN}(\mathbf{sk}, q)$ 
7:      $\text{TokenSet} = \text{TokenSet} \cup \{\mathbf{st}_q\}$ 
8:   return  $\mathbf{sk}, \mathbf{pp} = \{\mathbf{EDB} \cup \text{TokenSet}\}$ 

```

SEC.Encrypt. This routine is executed on the client’s end and is responsible for encrypting and offloading the actual data. As shown in Algorithm 7, the client uses its secret key $\mathbf{ke}$ to encrypt plaintext bits $\{x_1, \dots, x_p\}$ using an IND-CPA secure symmetric-key encryption scheme ($\Pi_{\text{sym}} = (\text{KEYGEN}, \text{ENCRYPT}_{\mathbf{ke}}, \text{DECRYPT}_{\mathbf{ke}})$), and returns a ciphertext $\{c_1, \dots, c_p\}$.

Algorithm 7 SEC.ENCRYPT

Input: $\mathbf{ke}, x_1, \dots, x_p$

Output: $c_1, \dots, c_p$

```

1: function SEC.ENCRYPT( $\mathbf{ke}, x_1, \dots, x_p$ )
2:   Encrypt input bits  $\{x_1, \dots, x_p\}$  using an IND-CPA secure symmetric-key encryption scheme
    $\Pi_{\text{sym}}$  with client secret-key  $\mathbf{ke}$ 
3:    $\{c_1, \dots, c_p\} \leftarrow \Pi_{\text{sym}}.\text{ENCRYPT}_{\mathbf{ke}}(x_1, \dots, x_p)$ 
4:   return  $\{c_1, \dots, c_p\}$ 

```

SEC.Evaluate. SEC.EVALUATE is executed by the server and is mainly responsible for computing a circuit (whose description is provided by the client) on encrypted inputs. This is summarized in Algorithm 9. We assume the client wants to compute an arbitrary Boolean circuit:

$$f_{\text{cir_depth}}(f_{\text{cir_depth}-1}(x_{\text{cir_depth}-1}, y_{\text{cir_depth}-1}), \dots, f_1(x_1, y_1))$$

where $f_j \in \{f_{\text{XOR}}, f_{\text{AND}}, f_{\text{OR}}\}$ and (x_j, y_j) are inputs to the j -th function f_j (where $1 \leq j \leq \text{cir_width}$; cir_width being the maximal width of the circuit) at depth i (for $1 \leq i \leq \text{cir_depth}$; cir_depth being the depth of the entire circuit). The server runs the SEC.EVALUATE algorithm that takes as input a circuit description f_{desc} , encrypted inputs $\{c_1, \dots, c_p\}$, and the public parameter $\mathbf{pp}$ (already offloaded to the server at the end of SEC.KEYGEN). It parses $\mathbf{pp}$ as $\{\mathbf{EDB}, \text{TokenSet}\}$ and then uses the encrypted inputs $\{c_1, \dots, c_p\}$ to retrieve the corresponding *search tokens* ($\mathbf{st}_{q_f}$) for all j functions at depth 1 of the circuit.

Next, for each of the j -th gate at depth i ($1 \leq i \leq \text{cir_depth}$), it invokes the CSSE.SEARCH algorithm using a specific search token $\mathbf{sk}$ and the $\mathbf{EDB}$ as input. The encrypted output obtained

Algorithm 8 ConstructQuery

Input: TokenSet, $\text{eval}_{j,i}, f_{\text{desc}}$ **Output:** Search Query

```
1: function ConstructQuery(TokenSet,  $\text{eval}_{j,i}, f_{\text{desc}}$ )
2:    $(\mathbf{w}_d^{j*,i+1}, b_{j*,i+1}) \leftarrow f_{\text{desc}}$   $\triangleright$  for a gate at width  $j$  and depth  $i$  in the circuit
3:   Compute a  $\text{token}_j$  using  $(\text{eval}_{j,i}, \mathbf{w}_d^{j*,i+1}, b_{j*,i+1})$ 
4:    $\text{xtoken}_{\mathbf{w}_d^{j*,i+1}, \mathbf{w}_{b_{j*,i+1}}, \text{eval}_{j,i}} \leftarrow \text{TokenSet}[\text{token}_j]$   $\triangleright$  for a circuit of width  $j$  at depth  $i$ , extract the
      search tokens in parallel
5:    $\text{xtoken}_j \leftarrow \text{xtoken}_j \cup \{\text{xtoken}_{\mathbf{w}_d^{j*,i+1}, \mathbf{w}_{b_{j*,i+1}}, \text{eval}_{j,i}}\}$ 
6:    $\text{st}_{q_f} \leftarrow \text{st}_{q_f} \cup \{\text{xtoken}_j\}$ 
7:   return  $\text{st}_{q_f}$ 
```

Algorithm 9 SEC.EVALUATE

Input: $f_{\text{desc}}, \{c_1, \dots, c_p\}, \text{pp}$ **Output:** eval

```
1: function SEC.EVALUATE( $f_{\text{desc}}, \{c_1, \dots, c_p\}, \text{pp}$ )
2:   Parse  $\text{pp} = \{\mathbf{EDB}, \text{TokenSet}\}$ 
3:   Retrieve search tokens  $\text{st}_{q_f}$  from TokenSet  $\triangleright \text{ConstructQuery}(\text{TokenSet}, \{c_1, \dots, c_n\}, f_{\text{desc}})$ 
4:   for  $i = 1$  to  $\text{cir\_depth}-1$  do  $\triangleright \text{cir\_depth}$  is depth of the circuit  $f_{\text{desc}}$ 
5:      $\text{eval}_{j,i} = \text{CSSE.SEARCH}(\text{sk}, \mathbf{EDB}, \text{st}_{q_f})$   $\triangleright$  for a circuit of width  $j$  at depth  $i$ , run  $j$  instances
      of CSSE.SEARCH in parallel
6:     Retrieve search tokens  $\text{st}_{q_f} \leftarrow \text{ConstructQuery}(\text{TokenSet}, \text{eval}_{j,i}, f_{\text{desc}})$ 
7:      $\text{eval} = \text{CSSE.SEARCH}(\text{sk}, \mathbf{EDB}, \text{st}_{q_f})$ 
8:   return eval to the client at the end of the protocol
```

(i.e. $\text{eval}_{j,i}$) is in turn used to retrieve the search tokens for the gate at depth $(i+1)$ and width j^* of the circuit by calling $\text{ConstructQuery}(\mathbf{w}_d^{j*,i+1}, \text{eval}_{j,i}, b_{j*,i+1})$. Note that $\mathbf{w}_d^{j*,i+1}$ is the randomly sampled “special” term for the *next* gate in the circuit which receives $\text{eval}_{j,i}$ as input, while $b_{j*,i+1}$ is the bit signifying whether $\text{eval}_{j,i}$ is the first or the second input to the next gate at depth $i+1$ and width j^* . Note that all this information is encapsulated in f_{desc} . The **ConstructQuery** routine retrieves the search token st_{q_f} . Again **CSSE.SEARCH** is called with the new search token query st_{q_f} and $\mathbf{EDB}$ as the input. This process continues till the last level of the circuit. Note, that each call to **CSSE.SEARCH** can be made in parallel for all j gates at a certain depth i of the circuit, since all functions at depth/level i are pairwise independent wrt. required inputs. The final encrypted evaluation eval returned by the last level of the circuit contains the encrypted bit corresponding to the actual output of the entire circuit evaluation. eval is returned to the client as the final encrypted output of the circuit f_{desc} .

Algorithm 10 SEC.DECRYPT

Input: ke, eval **Output:** result

```
1: function SEC.DECRYPT( $\text{ke}, \text{eval}$ )
2:    $\text{result} = \Pi_{\text{sym}}.\text{DECRYPT}_{\text{ke}}(\text{eval})$ 
3:   return result
```

SEC.Decrypt. The client runs the **SEC.DECRYPT** algorithm to decrypt the encrypted evaluation eval . The decrypted value result is equal to the evaluation of the circuit f_{desc} on $(x_1, \dots, x_p)$.

4 Proof of Correctness of SEC

The proof of correctness for SEC follows from the correctness of CSSE. The correctness of CSSE ensures that a conjunctive query $q = \mathbf{w}_1 \wedge \dots \wedge \mathbf{w}_n$ over an encrypted database satisfies the following relations (we refer to Section 2.1 for generic conjunctive SSE syntax):

$$\begin{aligned} \mathbf{sk} &\leftarrow \text{CSSE.KEYGEN}(\lambda) \\ \mathbf{EDB} &\leftarrow \text{CSSE.ENCRYPT}(\mathbf{sk}, \mathbf{DB}) \\ \mathbf{st}_q &= \text{CSSE.GENTOKEN}(\mathbf{sk}, q) \\ \mathbf{DB}(\mathbf{w}_1) \cap \dots \cap \mathbf{DB}(\mathbf{w}_n) &= \text{CSSE.SEARCH}(\mathbf{EDB}, \mathbf{st}_q) \end{aligned}$$

Proof. By deploying CSSE as a black-box, SEC generates the encrypted search index **EDB** specific to the function set $f = \{f_{\text{XOR}}, f_{\text{AND}}, f_{\text{OR}}\}$ supported by SEC. The encrypted search index **EDB** consists of keywords corresponding to encrypted input bits and documents that encapsulate the encrypted output of the function evaluation. It also invokes the CSSE.GENTOKEN algorithm that generates search tokens corresponding to the keywords (that map to encrypted input bits of a function). The search tokens are stored in a **TokenSet** and then $\mathbf{pp} = \{\mathbf{EDB} \cup \mathbf{TokenSet}\}$ is offloaded to the server. For evaluating a function $f(x, y)$, the function description f_{desc} (which is essentially a single binary function in this case) and encrypted inputs $\{c_1, c_2\}$ (corresponding to plaintext bits x and y respectively) are sent to the server. The server retrieves the search tokens from **TokenSet** and invokes CSSE.SEARCH protocol with the search token query $\mathbf{st}_{qf}$ and **EDB** as input. It retrieves a document from **EDB** that consists of the encrypted output of the computation, (denoted as **eval**) and sends it to the client. The client decrypts **eval** locally using the SEC.DECRYPT function and obtains the output bit **result** which is equal to $f(x, y)$. Correctness is hence a combination of correctness of CSSE, along with SEC specific mappings (Table 4 and Table 5) that ensure *exactly* one document being returned by CSSE.SEARCH, which is the *correct* result of respective gate evaluation.

Following the functionally correct evaluation of a single binary gate $f(\cdot, \cdot)$, the correct encrypted evaluation **eval** returned by SEC.EVALUATE for any arbitrary circuit f_{desc} over encrypted inputs $\{c_1, \dots, c_p\}$ can be asserted transitively. This is because f_{desc} is essentially viewable as a collection of several binary gates $f(\cdot, \cdot)$ which are functionally *correct* as established above, and thereby ensures that the output **eval** returned by SEC.EVALUATE on f_{desc} on decryption is equal to $f_{\text{desc}}(x_1, \dots, x_p)$ for any circuit f_{desc} and unencrypted input set $\{x_1, \dots, x_p\}$. Thereby, we conclude that for a functionally correct and exact conjunctive SSE scheme CSSE, a set of functions $f = \{f_{\text{XOR}}, f_{\text{AND}}, f_{\text{OR}}\}$, and encrypted values $\{c_1, \dots, c_p\}$ of input $\{x_1, \dots, x_p\}$, SEC is functionally correct, since for the following sequence of operations:

$$\begin{aligned} \mathbf{sk}, \mathbf{pp} &\leftarrow \text{SEC.KEYGEN}(\lambda) \\ \{c_1, \dots, c_p\} &\leftarrow \text{SEC.ENCRYPT}(\mathbf{ke}, x_1, \dots, x_p) \\ \mathbf{eval} &= \text{SEC.EVALUATE}(f_{\text{desc}}, c_1, \dots, c_p, \mathbf{pp}) \\ \mathbf{result} &= \text{SEC.DECRYPT}(\mathbf{ke}, \mathbf{eval}) \end{aligned}$$

result is the decrypted output of the evaluation of a circuit as specified by f_{desc} on encrypted inputs $c_1, \dots, c_p$, such that $\mathbf{result} = f_{\text{desc}}(x_1, \dots, x_p)$ (where, $x_1, \dots, x_p$ is the unencrypted input), i.e.,

$$\Pr[\mathbf{result} = f_{\text{desc}}(x_1, \dots, x_p)] = 1,$$

5 Complexity Analysis of SEC

Storage Overhead. The storage requirement of SEC depends upon the number of functions it supports along with the number of search tokens (input combinations) for every function. The final **DB** is collectively composed of three sub-databases (search indices) $\mathbf{DB} = \{\mathbf{DB}_{\text{AND}}, \mathbf{DB}_{\text{OR}}, \mathbf{DB}_{\text{XOR}}\}$ encrypted and offloaded to the server. As discussed in Section 1.2 and Section 3.2, each function-specific sub-database contains exactly *four* keywords²³, which are mapped to exactly *two* documents (out of four possible documents). The bulk of storage overhead comes from the set of k distinct “special” terms $\{\mathbf{w}_d^1, \mathbf{w}_d^2, \dots, \mathbf{w}_d^k\}$ (for a circuit with n binary gates, where $k \geq n$), for which SSE specific data structures occupy $O(k)$ space. Some constant number of *dummy* documents are also added to maintain a uniform frequency of each keyword in the final encrypted database. Concretely, there are exactly four keywords for one binary function, four documents (including dummy documents) per keyword, and k “special” terms. Total storage for three gates can be calculated as -

$$\text{Total Storage} = [(((4 + k) \times 4) \times 3) \cdot b] \text{ bytes} = O(k)$$

where b is a constant.

Computation and Communication Overhead. The evaluation time of SEC for computing arbitrary sized Boolean circuit over encrypted data scales linearly with the search time complexity of the underlying CSSE scheme, which in turn depends upon the depth of the circuit. The crux of SEC is to bypass explicit circuit evaluation as done in state-of-the-art encrypted computation schemes like FHE and leverage the extremely efficient encrypted search capability of a CSSE scheme to evaluate functions on encrypted inputs.

By constructing the mapping for all pairwise combinations in $\{\mathbf{w}_d^1, \mathbf{w}_d^2, \dots, \mathbf{w}_d^k\}$, multiple circuits can be executed securely without the need to refresh SEC’s data structures. This is achieved by ensuring ① no two gates in the *same* circuit share a “special” term, and ② two isomorphic gates in *different* circuits also do not share corresponding “special” terms. Hence, the client’s communication overhead is majorly because of the *one-time* setup, which is dominated by “special” terms (see storage analysis above). Consequently, randomly client-generated permutations of $\{\mathbf{w}_d^1, \mathbf{w}_d^2, \dots, \mathbf{w}_d^k\}$ are used to evaluate subsequent circuits. Overall, the communication complexity is bounded by $O(k)$.

6 Leakage Profile Description and Security Analysis

We analyze the security of SEC in this section. We follow a semi-honest adversarial setting for our security analysis where the remote server is assumed to be honest-but-curious. This implies the untrusted server follows the algorithmic specification exactly, but can also observe and record additional information for analysis. Using the ideal/real world paradigm, we establish formally that a probabilistic polynomial-time (PPT) simulator can simulate the view of the adversarial server in an indistinguishable manner given only the leakage profile of SEC.

CSSE Leakage Profile. SEC undoubtedly inherits *some* leakages from the underlying CSSE construction (Refer to Section 2.1, 2.3 for more details); we capture some inheritance through the CSSE specific leakage profile. Note that CSSE is an adaptively secure conjunctive SSE scheme against a semi-honest adversary $\mathcal{A}$. The leakage of CSSE is thus characterized by the leakage function $\mathcal{L}_{\text{CSSE}}$ which is an ensemble of the leakage functions for ENCRYPT, GENTOKEN and SEARCH individually, expressed as:

$$\mathcal{L}_{\text{CSSE}} = \{\mathcal{L}_{\text{CSSE}}^{\text{ENCRYPT}}, \mathcal{L}_{\text{CSSE}}^{\text{GENTOKEN}}, \mathcal{L}_{\text{CSSE}}^{\text{SEARCH}}\}$$

²³We consider this as $\mathcal{O}(1)$ overhead in our analysis.

SEC Leakage Profile. Given the above CSSE leakage functions, security of SEC can be analyzed using SEC leakage function $\mathcal{L}_{\text{SEC}}$ in the same adaptive semi-honest adversarial model. $\mathcal{L}_{\text{SEC}}$ is composed of three separate leakage functions for KEYGEN, ENCRYPT, and EVALUATE (as expressed below), that capture the leakage from SEC.KEYGEN, SEC.ENCRYPT, and SEC.EVALUATE execution respectively.

$$\mathcal{L}_{\text{SEC}} = \{\mathcal{L}_{\text{SEC}}^{\text{KEYGEN}}, \mathcal{L}_{\text{SEC}}^{\text{ENCRYPT}}, \mathcal{L}_{\text{SEC}}^{\text{EVALUATE}}\}$$

Informally, note that $\mathcal{L}_{\text{SEC}}$ subsumes $\mathcal{L}_{\text{CSSE}}$ plus leakages from SEC specific operations. Our security analysis thereby considers the security of SEC in presence of $\mathcal{L}_{\text{CSSE}}$ plus any leakage from any SEC specific operations.

Concrete Security of SEC_{OXT}. Note that $\mathcal{L}_{\text{CSSE}}$ is CSSE specific. For a concrete security analysis thereby, we provide the security analysis of SEC_{OXT}, i.e., SEC instantiated with OXT as the CSSE (Section 2.3)²⁴.

6.1 Leakages inherited (alt. rendered inconsequential) from OXT

We first capture the leakages inherited by SEC_{OXT} from the underlying CSSE construction OXT. We also note the rationale where SEC, by design, renders the said leakages inconsequential, wherein such leakage does not provide *any useful information to the adversary aiding it in compromising data privacy*. We prove this in Section 6.2.

- $N = \sum_{i=1}^d |W_i|$ - the total number of appearances of keywords in documents. The parameter N signifies an upper bound which is equivalent to the total size of the encrypted document collection. Leaking such a bound is unavoidable and is considered a trivial leakage in the literature of SSE.
- SP - size pattern of the queries i.e., the number of documents matching the **sterm** (least frequent keyword) in each query. Formally, $\text{SP} \in [d]^n$ and $\text{SP}[i] = |\text{DB}(\text{sterm}[i])|$. It leaks the number of documents satisfying the **sterm** in a query. In SEC_{OXT} this is always constant (equal to 2), since each keyword (encrypted input bit) maps to exactly four documents (encrypted output of function evaluated on the particular encrypted input) in **EDB**. Hence, SP is **inconsequential**.
- RP - result pattern of the queries or the indices of documents matching the entire conjunction. Formally, RP is vector of size n with $\text{RP}[i] = \text{DB}(\text{sterm}[i]) \cap \text{DB}(\text{xterm}[i])$ for each $i = 1, \dots, n$ where **xterm** refers to all the other keywords in the conjunctive query other than the **sterm**. It is the final output of the search query and is not considered a real leakage in the context of SSE. By the design of SEC_{OXT}, the result of a conjunctive query (binary function evaluation) is always a single document (single encrypted output bit) and hence this leakage is also **inconsequential**.
- IP - conditional intersection pattern, a $n \times n$ table defined as -

$$\text{IP}[i, j] = \begin{cases} |\text{DB}(s[i]) \cap \text{DB}(s[j])| & \text{if } i \neq j \text{ and } x[i] = x[j], \\ \phi & \text{otherwise.} \end{cases}$$

IP captures a leakage which occurs due to the specialised computation of **xtags** and storing a unique **xtag** value in the XSet for each (w-id) pair. Fundamentally in SEC, IP would capture the leakage which occurs when two distinct function evaluations have a common **xterm** (or second input) but different **sterm** (or first input) and there exists a document that satisfies both the **sterms**. In such a scenario the set of document indices matching both **sterms** is leaked (if no document matching both **sterms** exist then nothing is leaked). This is also **inconsequential** in

²⁴Analysis for SEC_{CONJ} follows suit.

SEC_{OXT} because in our construction **sterms** are nothing but dummy keywords that are mapped to all the documents present in the database.

- **EP** - Equality pattern indicates which queries have the equal “special” terms. This occurs due to the optimization technique devised in **OXT** in order to ensure sub-linear search complexity by filtering out the least frequent term (**sterm**) during the search. In SEC_{OXT}, since the frequency of all keywords is the same, the client chooses a different “special” term from a pool of dummy keywords ($\{\mathbf{w}_d^1, \mathbf{w}_d^1, \dots, \mathbf{w}_d^k\}$) for different queries (gate evaluation) on isomorphic gates in the circuit.. Hence, the adversary will not be able to infer any correlation for multiple gate evaluation over multiple (repeated/non-repeated) inputs, rendering this leakage **inconsequential**.

6.2 Security Analysis of SEC_{OXT}

Following up from the previous subsection, due to design choices in SEC_{OXT} (like uniform keyword frequency and selection of random “special” term for each query by the client), certain otherwise non-trivial leakages in **OXT** are rendered inconsequential in SEC_{OXT}. These leakages (viz. *size-pattern*, *result-pattern*, *equality-pattern*, *intersection pattern*) are essentially encapsulated by $\mathcal{L}_{\text{OXT}}^{\text{SEARCH}}$, and hence are also encapsulated by the SEC.EVALUATE algorithm (encapsulated by $\mathcal{L}_{\text{SEC OXT}}^{\text{EVALUATE}}$) considered below. We now state the main theorem.

Theorem 1 *Given that **OXT** is an adaptively secure CSSE scheme with respect to the leakage function $\mathcal{L}_{\text{OXT}} = \{\mathcal{L}_{\text{OXT}}^{\text{ENCRYPT}}, \mathcal{L}_{\text{OXT}}^{\text{GENTOKEN}}, \mathcal{L}_{\text{OXT}}^{\text{SEARCH}}\}$ against a polynomially-bounded adaptive adversary, SEC_{OXT} is also an adaptively secure encrypted computation framework with respect to the leakage function $\mathcal{L}_{\text{SEC OXT}} = \{\mathcal{L}_{\text{SEC OXT}}^{\text{KEYGEN}}, \mathcal{L}_{\text{SEC OXT}}^{\text{ENCRYPT}}, \mathcal{L}_{\text{SEC OXT}}^{\text{EVALUATE}}\}$, where the SEC_{OXT} instantiation encrypts an input $\{x_1, \dots, x_p\}$ using an IND-CPA secure symmetric-key encryption scheme to obtain corresponding encrypted bits $\{c_1, \dots, c_p\}$ over which a (publicly known) function f is evaluated, where f is composed of functions from $\{f_{\text{XOR}}, f_{\text{AND}}, f_{\text{OR}}\}$.*

Proof 1 *We give an extensive security analysis of SEC_{OXT} through formal proof of Theorem 1 next wrt. its data privacy guarantees.*

The security analysis of SEC_{OXT} (proof of Theorem 1) stems from the provable security guarantee of **OXT**. The proof outline is as such: we begin by inheriting the leakages from **OXT**, then point out the *additional* leakage in SEC_{OXT} (example: **ConstructQuery**), and thereafter establish the adaptive security guarantees of SEC_{OXT} through the simulation based security analysis. We show that SEC_{OXT} is secure against an adaptive semi-honest adversary $\mathcal{A}$, which has access to leakages from $\mathcal{L}_{\text{SEC OXT}}$. We build a simulator $\text{SIM} = \{\text{SIM}_{\text{KEYGEN}}, \text{SIM}_{\text{ENCRYPT}}, \text{SIM}_{\text{EVALUATE}}\}$ for SEC_{OXT} where the simulator emulates SEC_{OXT} execution just from the knowledge of public information and leakage $\mathcal{L}_{\text{SEC OXT}}$.

Leakage Cover. We briefly describe why each of the individual leakage components ($\mathcal{L}_{\text{SEC OXT}} = \{\mathcal{L}_{\text{SEC OXT}}^{\text{KEYGEN}}, \mathcal{L}_{\text{SEC OXT}}^{\text{ENCRYPT}}, \mathcal{L}_{\text{SEC OXT}}^{\text{EVALUATE}}\}$) are necessary for a simulator to produce correct results. Note that the public parameter **pp** captures N (or the total number of appearances of keywords in the database, i.e. the size of the **EDB**), and the size of search token set $|\text{TokenSet}|$. In **OXT**, the **EDB** comprises two data structures $\text{EDB} = \{\text{TSet}, \text{XSet}\}$. The simulator for SEC_{OXT} initializes the **XSet** and **TokenSet** to consist entirely of uniformly random elements from a *discrete log* hard group initially (while relying on the DDH assumption for indistinguishability of the real and simulated **XSet** and **TokenSet** entries). Additionally, the simulator for SEC_{OXT} can directly invoke the simulator for the adaptively secure **TSet** to simulate the **TSet** entries at **ENCRYPT**. Overall $\text{SIM} = \{\text{SIM}_{\text{KEYGEN}}, \text{SIM}_{\text{ENCRYPT}}, \text{SIM}_{\text{EVALUATE}}\}$ takes as input the leakage components as defined by $\mathcal{L}_{\text{SEC OXT}}^{\text{KEYGEN}}$, $\mathcal{L}_{\text{SEC OXT}}^{\text{ENCRYPT}}$, and $\mathcal{L}_{\text{SEC OXT}}^{\text{EVALUATE}}$. Note that N and $|\text{TokenSet}|$ is essentially learned

from the public parameter $\mathbf{pp}$, hence, $\mathcal{L}_{\text{SEC}_{\text{OXT}}}^{\text{KEYGEN}} = \perp$. Likewise, $\mathcal{L}_{\text{SEC}_{\text{OXT}}}^{\text{ENCRYPT}}$ reveals the length n of the ciphertext $\{c_1, \dots, c_n\}$, and $\mathcal{L}_{\text{SEC}_{\text{OXT}}}^{\text{EVALUATE}}$ reveals $\{\mathbf{SP}, \mathbf{RP}, \mathbf{EP}, \mathbf{IP}\}$ and leakages from the **ConstructQuery** function. Using these leakages and the public parameter the SIM then produces the ciphertext $\{c_1, \dots, c_n\}$ which is indistinguishable from the encrypted output returned by the original $\text{SEC}_{\text{OXT}}.\text{ENCRYPT}$ algorithm on an input $\{x_1, \dots, x_n\}$.

Simulating $\text{SEC}_{\text{OXT}}.\text{KEYGEN}$. We observe that, $\text{SEC}_{\text{OXT}}.\text{KEYGEN}$ comprises of the **GENDB**, **OXT.ENCRYPT** (CSSE.ENCRYPT), **OXT.GENTOKEN** (CSSE.GENTOKEN) (this phase is encapsulated in the **OXT.SEARCH** protocol in [15], and is entirely executed by the client).

We first focus on the simulation of **GENDB**. Without loss of generality, we extract the search token generation phase (**OXT.GENTOKEN**) and store all possible search tokens in **TokenSet** during $\text{SEC}_{\text{OXT}}.\text{KEYGEN}$. Pseudo-random outputs generated by pseudo-random functions (PRF) and group operations are inserted into **TSet** using **TSet.SETUP** [15] and **XSet** respectively during **OXT.ENCRYPT**, and are thus simulable using unkeyed random functions. Note that, **GENDB** routine is executed on the client side. Hence, the adversarial server learns no information from the **GENDB** execution itself and thus the leakage from **GENDB** can be expressed as null.

$$\mathcal{L}^{\text{GENDB}} = \perp,$$

Thus, the simulator $\text{SIM}_{\text{KEYGEN}}$ can exactly simulate **GENDB** execution straightforwardly. Subsequently, the **OXT.ENCRYPT** is invoked with the plain **DB** generated by **GENDB**, and can thereby be simulated in a manner similar as in [15]. Moreover, since the **OXT.ENCRYPT** algorithm is executed in a black-box manner, the leakage $\text{SEC}_{\text{OXT}}.\text{KEYGEN}$ is same as the **OXT.ENCRYPT** executed over **DB**. Also, **OXT.GENTOKEN** phase is invoked on all possible queries q of keywords (alphanumeric translations of encrypted input bits) in **DB**. This algorithm is also used a black-box and is entirely executed by the client. Thus, the leakage from **EDB** used for simulation is:

$$\mathcal{L}_{\text{SEC}_{\text{OXT}}}^{\text{EDB}} = \{\mathcal{L}_{\text{OXT}}^{\text{ENCRYPT}}(\mathbf{DB}), \mathcal{L}_{\text{OXT}}^{\text{GENTOKEN}}(q)\},$$

Finally, the **TSet SETUP** execution is simulable from already known public information (the size of the database $|\mathcal{W}| = N$ is known), as expressed below, combined with unkeyed random functions, just like the **TSet** simulator (as discussed in the original paper [15]).

$$\mathcal{L}_{\text{SEC}_{\text{OXT}}}^{\text{KEYGEN, TSet}} = N,$$

Combined all, the simulator for **KEYGEN** ($\text{SIM}_{\text{KEYGEN}}$) simulates $\text{SEC}_{\text{OXT}}.\text{KEYGEN}$ with access to following the leakage

$$\mathcal{L}_{\text{SEC}_{\text{OXT}}}^{\text{KEYGEN}} = \{\mathcal{L}^{\text{GENDB}}, \mathcal{L}_{\text{SEC}_{\text{OXT}}}^{\text{EDB}}, \mathcal{L}_{\text{SEC}_{\text{OXT}}}^{\text{KEYGEN, TSet}}\},$$

by using the respective simulators from [15].

Simulating $\text{SEC}_{\text{OXT}}.\text{Encrypt}$. For simulating $\text{SEC}_{\text{OXT}}.\text{ENCRYPT}$, the simulator $\text{SIM}_{\text{ENCRYPT}}$ observes $\mathcal{L}_{\text{SEC}_{\text{OXT}}}^{\text{ENCRYPT}}$ which is equal to the length of the ciphertext $\{c_1, \dots, c_n\}$.

$$\mathcal{L}_{\text{SEC}_{\text{OXT}}}^{\text{ENCRYPT}} = |c_1, \dots, c_n|,$$

The $\text{SEC}_{\text{OXT}}.\text{ENCRYPT}$ algorithm invokes an IND-CPA secure symmetric-key encryption scheme which is used to encrypt an input bit $\{x_1, \dots, x_n\}$. The $\text{SIM}_{\text{ENCRYPT}}$ produces a ciphertext $\{c_1, \dots, c_n\}$ corresponding to the input only with the information from $\mathcal{L}_{\text{SEC}_{\text{OXT}}}^{\text{ENCRYPT}}$ and its state (ssim). The ciphertext thus produced by $\text{SIM}_{\text{ENCRYPT}}$ is indistinguishable from the ciphertext produced by $\text{SEC}_{\text{OXT}}.\text{ENCRYPT}$ in the real scheme (involving the secret key $\mathbf{ke}$; Algo. 7).

Simulating SEC_{OXT} .Evaluate. The EVALUATE function takes as input encrypted bits $\{c_1, \dots, c_n\}$ and retrieves *search tokens* from the TokenSet using the encrypted input bits (as input to the ConstructQuery subroutine). It then invokes the OXT.SEARCH function using the search tokens and gets an encrypted bit as output. This process is repeated for all gates at every depth of the circuit being evaluated. The ConstructQuery subroutine leaks essentially no information to the server about the input bits and the output of the function evaluation, since they are encrypted using an IND-CPA symmetric-key encryption algorithm. It just leaks the *access pattern* of the xtokens ($\text{AP}_{\text{xtoken}}$) retrieved from TokenSet corresponding to a “special” term for that particular function/query and the keyword corresponding to the input bit to the function. However, we argue that this *access pattern* leakage of search tokens cannot be exploited by the adversarial server to gain any correlation between the inputs to a function and its corresponding output over multiple circuit evaluations. The reason is because each xtoken is computed using a client controlled “special” term that is selected at random by the client. The number of possible permutation of such “special” terms for a circuit with n gates is upper bounded by $O(k)$. Therefore, considering the following leakage for ConstructQuery :

$$\mathcal{L}^{\text{ConstructQuery}} = \text{AP}_{\text{xtoken}},$$

Note that xtoken is dependent on the “special” term assigned to each gate (see Algorithm 8, wherein the $\mathbf{w}_d^{j^*, i+1}$ represents the contribution of the “special” term to $\text{AP}_{\text{xtoken}}$). Combined with the leakage incurred during the actual search, we can write the overall leakage $\mathcal{L}_{\text{SEC}_{\text{OXT}}}^{\text{EVALUATE}}$ required for simulation as:

$$\mathcal{L}_{\text{SEC}_{\text{OXT}}}^{\text{EVALUATE}} = \{\mathcal{L}^{\text{ConstructQuery}}, \mathcal{L}_{\text{OXT}}^{\text{SEARCH}}(\text{EDB}, st_q)\},$$

It is crucial for SEC that repeated re-execution of the same circuit does not assign the same “special” term more than once to the same gate; this ensures that $\mathbf{w}_d^{j^*, i+1}$ input to ConstructQuery is always fresh, transitively implying that the leakage $\text{AP}_{\text{xtoken}}$ does not offer any correlation to the server. This is guaranteed by the PRP security as a client permutes the choice of “special” terms using a PRP. To simulate this, the $\text{SIM}_{\text{EVALUATE}}$ maintains a history array- ArrHist - that stores the *ordering* of (randomly permuted) “special” terms used in previous evaluations. In order to simulate randomly permuted “special” terms for the circuit, the simulator randomly²⁵ selects a “special” term from the client permuted list of “special” terms, SList . After choosing a random “special” term from SList , it checks whether the same entry matches with any record in ArrHist . If yes, it discards it and samples a fresh “special” term and repeats the check. Or else, if there is no match found in ArrHist it uses the current “special” term permutation for the current circuit evaluation. This ensures no “special” term is repeated for multiple evaluation of the same/different circuit. In case a fresh permutation cannot be found, the simulator bails out: the circuit cannot be simulated securely anymore, and requires a fresh SEC.KEYGEN and SEC.ENCRYPT before resuming SEC.EVALUATE . Thus, by the characterization of $\mathcal{L}^{\text{ConstructQuery}}$ performed here, leakage from ConstructQuery does not undermine security guarantees of SEC as long as fresh permutations are used.

Also note that by the simulation security guarantee of OXT, SEC_{OXT} is secure against $\mathcal{L}_{\text{SEC}_{\text{OXT}}}^{\text{EVALUATE}}$. We prove the same now. Let a client evaluate a circuit of size n . The client sends the encrypted input bits to SEC.EVALUATE . A two-input function evaluation is translated to a three-keyword conjunctive query, where the first keyword is a “special” term (chosen randomly by the client), the second keyword corresponds to the first input bit, and the third keyword to the second input bit. The client maintains a state of record of all “special” terms used. For a circuit with

²⁵In the real scheme, this permutation is generated by a keyed PRP by the client.

n gates, the client chooses k “special” terms, which can be permuted and selected in ways upper bounded by $O(k)$. The client sends a set of “special” terms to be used at each gate in a circuit. Since the “special” terms are used to fetch the records from memory, permuting “special” terms for consecutive (same) gate evaluation ensures no repetition of the same “special” term hence, the same memory location is not accessed twice. Thus, *using the same look-up table that is encrypted and offloaded to the server once during SEC.KEYGEN phase, SEC can evaluate any arbitrary Boolean circuit multiple times.* This is guaranteed both in a single circuit across multiple gate evaluation as well as across multiple circuit evaluation. This can be stated formally as the following corollary of Theorem 1.

Corollary 1 (Reusability of Lookup Table) *Given that SEC_{OXT} encrypts an input $x_1, \dots, x_p$ using an IND-CPA secure symmetric-key encryption scheme to obtain corresponding encrypted bits $c_1, \dots, c_p$ that is used as an input to a (publicly known) function f , and all information leaked from the underlying CSSE.SEARCH (OXT.SEARCH) phase is encapsulated by $\mathcal{L}_{\text{OXT}}^{\text{SEARCH}}$, SEC_{OXT} ensures reusability of the same look-up table for multiple (similar/different) gate evaluations without leaking any extra information than that encapsulated by $\mathcal{L}_{\text{SEC}_{\text{OXT}}}$, while guaranteeing input and output data privacy from semantic security guarantees of an IND-CPA secure encryption scheme.*

We can explain this further with the help of the following two cases:

Case-I: This case corresponds to executions of encrypted gates at all but the *first* level of the circuit (lines 4–6 in Algorithm 9), whose search tokens are generated recursively by the output of the previous level. For such gates, if the same circuit is evaluated more than once, fresh permutations of the “special” terms leaks a fresh $\text{AP}_{\text{xtoken}}$ to the server for each gate. Also note that, by definition, fresh assignment of “special” terms also leaks fresh leakages $\{\text{SP}, \text{RP}, \text{EP}, \text{IP}\}$ that cannot be correlated across executions; we briefly summarize how these leakages are rendered inconsequential by design of SEC:

- **Access Pattern of “special” term:** Two queries having the *same* “special” term can be correlated by the server by the same set of $\text{sval}_{\mathbf{w}_d, D_j}$ returned. We emphasize that the server learns not the plaintext alphanumeric value of $\mathbf{w}_d$, but rather the fact that two queries share the same “special” term. Data privacy of $\mathbf{w}_d$ thus still remains intact because “special” terms do not represent the actual input bits, they are client controlled and randomly sampled alphanumeric strings. Still, assignment of fresh “special” terms ensures two queries do not have the same “special” term and thus prevents the server from creating meaningful correlations through access pattern leakage.
- **Access Pattern of “actual” keywords:** Two queries having a common document matched to their “special” term and the same cross term tuples (for example, $(\mathbf{w}_d, \mathbf{w}_i)$) are leaked since the same $\text{Token}_{\mathbf{w}_d, \mathbf{w}_i, D_j}$ is generated for both queries. As before, the server can not learn the underlying plaintext alphanumeric value of $\mathbf{w}_i$; it can simply correlate same cross-terms across two queries. Data privacy of $\mathbf{w}_i$ thus still remains intact. Additionally, note that choice of fresh “special” term also prevents the server from creating meaningful correlations through access pattern leakage.

*Note. In a generic CSSE protocol, this form of leakage is referred to as the Intersection Pattern leakage (IP). Several works have proposed improvements to CSSE schemes that eliminate this leakage, albeit at the expense of quadratic storage overhead [44, 48]. We highlight that SEC treats the encrypted search functionality of any CSSE scheme as a black box. Consequently, SEC is amenable to **suppressing** these leakages, depending on the underlying CSSE used.*

- **Query Result Pattern:** Two queries having the same result (i.e. $\text{sval}_{\mathbf{w}_d, D_j}$) can be correlated by the server. Data privacy of D_j still remains intact. As before, fresh “special” assignment prevents the server from creating meaningful correlations through query result pattern leakage.

Overall, once the “special” terms are freshly assigned, gate executions are simulated through simulation of OXT.SEARCH using a conjunctive query of 3 keywords (see **Illustrative Example 1** for referential examples) using the leakages $\{\text{SP}, \text{RP}, \text{EP}, \text{IP}, \text{AP}_{\text{xtoken}}\}$. Thus, through leakage of fresh $\{\text{SP}, \text{RP}, \text{EP}, \text{IP}, \text{AP}_{\text{xtoken}}\}$ in re-executions, combined with simulation security guarantee of OXT.SEARCH, the server cannot distinguish between two identical gate evaluations with overwhelmingly high probability.

Case-II: This case corresponds to executions of the encrypted gates at the *first* level of the circuit (line 3 in Algorithm 9). The only difference with Case-I is the manner of retrieving search tokens: here the tokens are retrieved based on the encrypted input bits, and based on the “special” terms assigned to the first level. Like with Case-I, gate evaluation is simulated through OXT.SEARCH using a conjunctive query of 3 keywords, and fresh leakages $\{\text{SP}, \text{RP}, \text{EP}, \text{IP}, \text{AP}_{\text{xtoken}}\}$ ensure the server cannot distinguish between two identical gate evaluations with overwhelmingly high probability.

We detailed how $\text{SIM} = \{\text{SIM}_{\text{KEYGEN}}, \text{SIM}_{\text{ENCRYPT}}, \text{SIM}_{\text{EVALUATE}}\}$ simulates the entire SEC protocol based on the leakage function $\mathcal{L}_{\text{SEC}_{\text{OXT}}}$. We further establish how the leakages are governed by permutation of “special” terms (guaranteed by PRP security). Therefore, the output of SEC.EVALUATE is indistinguishable from random by the security guarantees of an IND-CPA secure symmetric-key encryption scheme. Since OXT is proven simulation secure it follows from its simulation security guarantee that a PPT adversary $\mathcal{A}$ has no additional advantage over the real experiment. This implies the **Real** experiment of SEC (Algorithm 3) is indistinguishable from the **Ideal** experiment (Algorithm 4), and proves Theorem 1.

7 Statistical Analysis of Leakage Due to Reusability

We demonstrated a proof of computational indistinguishability (from an adversarial perspective) that establishes the secure reusability of SEC’s lookup tables in Section 6. In this section, we provide statistical analysis of the leakage from SEC’s lookup tables to validate the same. For this, we closely follow the non-interference security notion well established in several side-channel analysis paradigms [4, 22].

Abstractly, the *non-interference property* ensures no sensitive information flow to the output of a system, given the system’s inputs. In context of SEC, non-interference between inputs to SEC.EVALUATE and $\mathcal{L}_{\text{SEC}}^{\text{EVALUATE}}$ (i.e. the observable leakage) translates directly to the server’s inability to infer (with statistical significance) anything about the inputs purely from $\mathcal{L}_{\text{SEC}}^{\text{EVALUATE}}$. Concretely, non-interference can be defined as [4, 22]:

Definition 1 (Non-Interference) *For a probabilistic program P , consider the set of secret (“high”) inputs as H , the set of public (“low”) inputs as L , and the output as $\mathcal{L}$. Then, P is said to be non-interfering if and only if the mutual information $\mathcal{I}(\mathcal{L}; H \mid L) = 0$.*

In other words, this information-theoretic definition captures the mutual information (or the mutual dependence) of $\mathcal{L}$ and H (or the critical, secret input to P), given knowledge of non-secret L . P is considered to be non-interfering if variations in H do not (statistically) affect $\mathcal{L}$ (given knowledge of L). This is captured by the mutual information (conditioned on L) being 0. However, estimating conditional mutual information in an information-theoretic setting is

a difficult problem in general. Thus, a slightly “relaxed” definition for non-interference can be used instead [22]:

Definition 2 (“Relaxed” Non-Interference) *For a probabilistic program P , consider the set of secret (“high”) inputs as H , the set of public (“low”) inputs as L , and the output as $\mathcal{L}$. Then, P is said to be non-interfering if the marginal distribution of $\mathcal{L}$ is independent of the distribution of H .*

Concretely, from the point of view of reusability in SEC (as in Theorem 1), the random choice of “special” terms by the client leads to a computationally indistinguishable view of the server for repeated evaluations. From the perspective of non-interference definitions established, given a query executed in SEC.EVALUATE, the “high” inputs H correspond to the actual tokens that map inputs to the function being evaluated, while the “low” input is the “special” term (that does not participate in actual functional evaluation). Evidently, $\mathcal{L}$ is then essentially the leakage observed from SEC’s evaluate phase (i.e. $\mathcal{L}_{\text{SEC}}^{\text{EVALUATE}}$).

To establish statistical independence between H and $\mathcal{L}$, we rely on Welch’s t-test (that is naturally applied when two populations have unequal variances). In our experiments, we allow the server to learn memory access patterns wrt. the aforementioned leakage profile of SEC.EVALUATE. The null hypothesis (for a two-tailed test) then tests whether the population means for two evaluations of SEC (while reusing SEC’s data structures) are indistinguishable. We initialize the “special” terms during SEC.KEYGEN as usual, and the client controls the permutations of the same over the execution of 1 million queries (i.e. a circuit consisting of 1 million gates). Our α value (i.e. the probability of incorrectly rejecting the null hypothesis when it is instead true) is 1%. Empirically, we observe a t-statistic of -1.7321 and a p-value of 0.0832 . We hence conclude that there is not sufficient evidence to reject the null hypothesis. In other words, statistically, the server’s view (given SEC.EVALUATE’s leakage profile) of reusable executions in SEC is statistically indistinguishable from the execution of a randomly sampled circuit of the same size (for client-controlled permutation of “special” terms). This statistically establishes non-interference of H in the leakage $\mathcal{L}_{\text{SEC}}^{\text{EVALUATE}}$.

8 Experimental Results

In this section, we report on a prototype implementation of SEC_{OXT} and SEC_{CONJFILTER}, and compare it with a prototype implementation of the TFHE library [21], which implements an efficient and fast gate-by-gate bootstrapping [20].

Implementation Details. Our prototype implementations are developed in C++ and we use Redis as the database backend. More specifically, we realize all PRF operations using AES-256 in counter mode, BLAKE3 hash function for computing all hash operations, and all group operations over the elliptic curve Curve25519 [7].

Platform. For our experiments, we used a *single* node with 64-bit Intel Xeon Silver 4214R v4 3.27GHz processors, running Ubuntu 20.04.4 LTS, with 128GB RAM and 1TB SSD hard disk.

Evaluation Of Storage Overhead. As discussed in Section 5, the storage required for SEC scales with the number of keyword-document pairs and the number of search tokens for a particular function. In our implementations, the server storage required for SEC_{OXT} to store TSet²⁶ and XSet²⁷ and the TokenSet is around 43 KB while that for SEC_{CONJFILTER} is 26 KB. We thus note that SEC is highly optimized and scalable with significantly fewer storage requirements than state-of-the-art FHE schemes. Table 7 offers a quantifiable comparison.

²⁶OXT specific data structure to store inverted index for the “special” term.

²⁷OXT specific data structure to check presence of “cross” terms in respective document identifiers.

Table 7: Storage Overhead comparison (in MB) of SEC with existing FHE schemes in literature. Storage overhead of FHE scheme typically indicates the bootstrapping key size whereas for SEC_{OXT} and $\text{SEC}_{\text{CONJFILTER}}$ it implies the size of the encrypted search index stored at the cloud server. Here, $k = 50000$, implying up to 50000 possible re-executions of a circuit with size $n = k$ with a single setup.

Scheme	Storage Overhead (in MB)
Gentry et. al.[32]	3700
Gentry et. al.[28]	2300
Halevi et. al.[38]	1600
Ducas et. al.[25]	1000
Chillotti et. al.[20]	24
$\text{SEC}_{\text{CONJFILTER}}$ (This work)	0.449
SEC_{OXT} (This work)	0.098

Evaluation of Computation time. The evaluation time of SEC_{OXT} for computing arbitrary sized Boolean circuit over encrypted data scales linearly with the search time complexity of OXT times some constant which depends upon the depth of the circuit. The time required to retrieve the documents corresponding to a conjunctive query scales with the least frequent keyword in the query in OXT. Since the database (search index) in SEC is extremely small, the time taken by OXT.SEARCH is significantly less. Hence, the average time required by SEC_{OXT} is around 10 milliseconds for one binary function evaluation which is remarkably fast. On a similar note $\text{SEC}_{\text{CONJFILTER}}$ scales with the search complexity of CONJFILTER, which is dependent on the least frequent conjunct in the query. Notably $\text{SEC}_{\text{CONJFILTER}}$ exhibits even faster performance with an average evaluation time of 40 microseconds for one binary function evaluation. Our experimental results validate that SEC is highly efficient and extremely fast while evaluating arbitrary Boolean functions over encrypted data. Figure 4 compares the execution time of SEC_{OXT} and $\text{SEC}_{\text{CONJFILTER}}$ with different TFHE backends for varying depth of circuits.

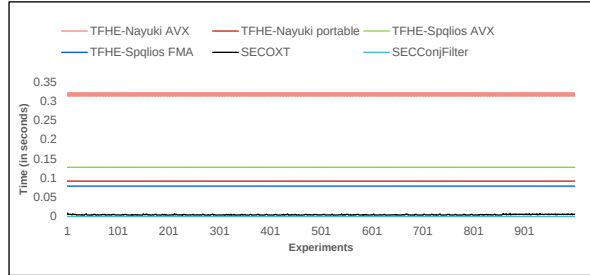


Figure 3: Time taken (in seconds) for 1000 invocations of SEC_{OXT} and $\text{SEC}_{\text{CONJFILTER}}$ against different TFHE backends.

Comparison With FHE. We compare SEC_{OXT} and $\text{SEC}_{\text{CONJFILTER}}$ with different variations of TFHE in Figure 3. One variation is Nayuki portable (non-AVX) and AVX builds, which implement very efficient versions of Fast Fourier Transform. Another back-end family is sqlios AVX and sqlios FMA back-ends, which are efficient assembly implementations of ring operations. It is observed from Figure 3 that SEC_{OXT} is $10^3 \times$ and six to seven times faster; $\text{SEC}_{\text{CONJFILTER}}$

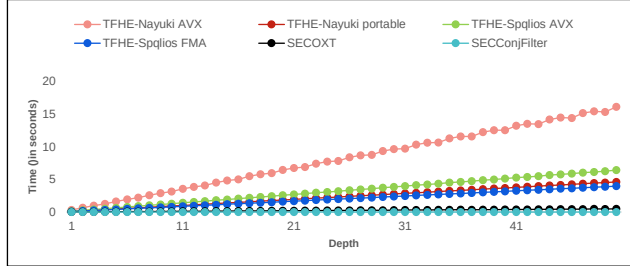


Figure 4: Time taken (in seconds) for different circuit depths of SEC_{OXT} and $\text{SEC}_{\text{CONJFILTER}}$ against different TFHE backends.

Table 8: Time taken (in minutes) and Storage overhead (in MB) for evaluation of AES-128 circuit and Maxpool function (AlexNet) by SEC_{OXT} and $\text{SEC}_{\text{CONJFILTER}}$ against different TFHE backends. For both circuits, SEC allows for $k = 50000$ re-evaluations with a single setup. A GC setup can evaluate these circuits once per setup; we hence omit concrete comparisons with it.

Scheme	Time (in minutes)		Storage (in MB)
	AES-128	Maxpool Function	
TFHE-Nayuki Portable	336.03	2920.52	24
TFHE-Nayuki AVX	179.02	1441.18	24
TFHE-Spqlios AVX	57.37	523.62	24
TFHE-Spqlios FMA	41.87	349.30	24
$\text{SEC}_{\text{CONJFILTER}}$ (This work)	1.02	6.18	0.449
SEC_{OXT} (This work)	6.57	48.17	0.098

is $10^6 \times$ and $10^3 \times$ faster than the portable TFHE backend and the fastest (non-portable) TFHE backend Spqlios AVX, respectively. Figure 4 compares the increase in execution time with an increase in the depth of the circuit. Both instantiations of SEC outperforms the fastest TFHE backend using Spqlios AVX optimization for function evaluation of arbitrary depth.

We showcase SEC’s scalability for functions with multi-bit inputs by using it for encrypted evaluation of (i) the entire AES-128 circuit (with XOR/AND/NOT-gate count of 25124/6800/1692) and (ii) three max-pooling layers of AlexNet architecture²⁸. This requires no extra storage (since we still only require storage for three extra gates), and the performance figures (as well as a comparison with Torus-FHE) are described in Table 8. For both circuits, a (non-parallelized) implementation of SEC_{OXT} outperforms a (non-parallelized) implementation of Torus-FHE by six to seven orders of magnitude, while a (non-parallelized) implementation of $\text{SEC}_{\text{CONJFILTER}}$ shows an improvement of $10^3 \times$ in computation time (we expect the relative comparison to remaining unchanged with parallelization and additional hardware/software-level optimizations). SEC_{OXT} requires around $250 \times$ less storage while $\text{SEC}_{\text{CONJFILTER}}$ requires $50 \times$ times less storage, which are remarkably less. These results clearly showcase the efficiency and scalability of SEC for circuits with multi-bit inputs.

9 Discussion

We introduced SEC, a lookup-based framework for evaluating arbitrary Boolean circuits over symmetrically encrypted data. Unlike traditional FHE, which relies on heavy algebraic struc-

²⁸KSH17 Imagenet classification with deep convolutional neural networks

tures to support computation on public ciphertexts, SEC models each gate as a truth table and realizes gate evaluation via encrypted lookups. This avoids bootstrapping, yielding large gains in computation time and storage efficiency, while still supporting reusable evaluations through client-controlled permutations of encrypted lookup tables. As demonstrated empirically, SEC outperforms traditional FHE schemes significantly, both in terms of computation time and storage requirements, when operating over symmetrically encrypted data. Compared to Garbled Circuits, SEC achieves similar lookup-level efficiency but crucially enables reusability across multiple evaluations, something fundamentally absent in GCs, while operating in settings where circuit privacy is not required. Thus, SEC occupies a practically important middle ground: significantly more efficient and scalable than FHE for symmetric-key settings, while offering stronger reusability than GCs.

The algebraic structure underlying FHE allows it to operate over publicly encrypted data, and we leave it as an interesting open question to extend the lookup-based approach underlying SEC to computing over publicly encrypted data. We note, however, that in many practical applications (e.g., querying over outsourced encrypted databases), it suffices to support evaluation of arbitrary Boolean circuits over symmetrically encrypted data, since the data owner is also the primary entity querying the (encrypted) data after outsourcing it to an untrusted server for storage and processing. Indeed, this setting motivates the entire literature on SSE [47, 24, 18, 15], albeit for restricted classes of functions. To the best of our knowledge, our work is the first to establish the possibility of supporting arbitrary Boolean circuit evaluation efficiently over encrypted data using purely symmetric-key encryption techniques on top of lookup-based gate evaluation. Indeed, as demonstrated by our theoretical analysis and practical evaluation, the usage of purely symmetric-key primitives is what enables the highly desirable efficiency and compactness guarantees of SEC, allowing it to scale over extremely large symmetrically encrypted circuits while outperforming FHE. Overall, SEC establishes the first evidence that purely symmetric-key, lookup-based primitives can scale to large circuits while providing strong efficiency, compactness, and reusability guarantees, thereby addressing a critical gap between FHE and GCs in outsourced computation.

References

- [1] Al Badawi, A., Bates, J., Bergamaschi, F., Cousins, D.B., Erabelli, S., Genise, N., Halevi, S., Hunt, H., Kim, A., Lee, Y., et al.: Openfhe: Open-source fully homomorphic encryption library. In: WAHC (2022)
- [2] Alperin-Sheriff, J., Peikert, C.: Practical bootstrapping in quasilinear time. In: Advances in Cryptology—CRYPTO 2013: 33rd Annual Cryptology Conference, Santa Barbara, CA, USA, August 18–22, 2013. Proceedings, Part I (2013)
- [3] Alperin-Sheriff, J., Peikert, C.: Faster bootstrapping with polynomial error. In: Advances in Cryptology—CRYPTO 2014: 34th Annual Cryptology Conference, Santa Barbara, CA, USA, August 17–21, 2014, Proceedings, Part I 34 (2014)
- [4] Barthe, G., Belaïd, S., Dupressoir, F., Fouque, P.A., Grégoire, B., Strub, P.Y.: Verified proofs of higher-order masking. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques. pp. 457–485. Springer (2015)
- [5] Bellare, M., Hoang, V.T., Rogaway, P.: Foundations of garbled circuits. In: the ACM Conference on Computer and Communications Security, CCS’12, Raleigh, NC, USA (2012)

- [6] Ben-Efraim, A., Lindell, Y., Omri, E.: Efficient scalable constant-round MPC via garbled circuits. In: *Advances in Cryptology - ASIACRYPT 2017 - 23rd International Conference on the Theory and Applications of Cryptology and Information Security, Proceedings, Part II* (2017)
- [7] Bernstein, D.J.: Curve25519: New diffie-hellman speed records. In: *Public Key Cryptography - PKC. Lecture Notes in Computer Science* (2006)
- [8] Boemer, F., Kim, S., Seifu, G., DM de Souza, F., Gopal, V.: Intel hexl: accelerating homomorphic encryption with intel avx512-ifma52. In: *WAHC* (2021)
- [9] Boneh, D., Sahai, A., Waters, B.: Functional encryption: Definitions and challenges. In: *Theory of Cryptography: 8th Theory of Cryptography Conference, TCC 2011, Providence, RI, USA, March 28-30, 2011. Proceedings 8* (2011)
- [10] Brakerski, Z., Gentry, C., Halevi, S.: Packed ciphertexts in lwe-based homomorphic encryption. In: *Public-Key Cryptography-PKC 2013* (2013)
- [11] Brakerski, Z., Gentry, C., Vaikuntanathan, V.: (leveled) fully homomorphic encryption without bootstrapping. *ACM Transactions on Computation Theory (TOCT)* (2014)
- [12] Brakerski, Z., Vaikuntanathan, V.: Efficient fully homomorphic encryption from (standard) LWE. In: *IEEE 52nd Annual Symposium on Foundations of Computer Science, FOCS 2011, Palm Springs, CA, USA, October 22-25, 2011* (2011)
- [13] Brakerski, Z., Vaikuntanathan, V.: Efficient fully homomorphic encryption from (standard) lwe. *SIAM Journal on computing* (2014)
- [14] Cash, D., Jaeger, J., Jarecki, S., Jutla, C.S., Krawczyk, H., Rosu, M., Steiner, M.: Dynamic searchable encryption in very-large databases: Data structures and implementation. In: *NDSS 2014* (2014)
- [15] Cash, D., Jarecki, S., Jutla, C., Krawczyk, H., Roşu, M.C., Steiner, M.: Highly-scalable searchable symmetric encryption with support for boolean queries. In: *CRYPTO* (2013)
- [16] Cash, D., Jarecki, S., Jutla, C.S., Krawczyk, H., Rosu, M., Steiner, M.: Highly-scalable searchable symmetric encryption with support for boolean queries. In: *CRYPTO 2013* (2013)
- [17] Chang, Z., Xie, D., Li, F.: Oblivious ram: A dissection and experimental evaluation. *Proceedings of the VLDB Endowment* (2016)
- [18] Chase, M., Kamara, S.: Structured encryption and controlled disclosure. In: *ASIACRYPT 2010* (2010)
- [19] Cheon, J.H., Kim, A., Kim, M., Song, Y.: Homomorphic encryption for arithmetic of approximate numbers. In: *ASIACRYPT 2017* (2017)
- [20] Chillotti, I., Gama, N., Georgieva, M., Izabachene, M.: Faster fully homomorphic encryption: Bootstrapping in less than 0.1 seconds. In: *ASIACRYPT 2016* (2016)
- [21] Chillotti, I., Gama, N., Georgieva, M., Izabachène, M.: Tfhe: Fast fully homomorphic encryption library (2019)

- [22] Clark, D., Hunt, S., Malacaria, P.: Quantified interference: Information theory and information flow. In: Workshop on Issues in the Theory of Security (WITS'04) (2004)
- [23] Clearinghouse., P.R.: Chronology of data breaches. <https://privacyrights.org/data-breaches> (2024)
- [24] Curtmola, R., Garay, J., Kamara, S., Ostrovsky, R.: Searchable symmetric encryption: improved definitions and efficient constructions. In: ACM CCS (2006)
- [25] Ducas, L., Micciancio, D.: Fhew: bootstrapping homomorphic encryption in less than a second. In: EUROCRYPT 2015 (2015)
- [26] El-Yahyaoui, A., Kettani, M.D.E.E.: A verifiable fully homomorphic encryption scheme for cloud computing security. CoRR (2018)
- [27] Gentry, C.: Fully homomorphic encryption using ideal lattices. In: Proceedings of the forty-first annual ACM symposium on Theory of computing (2009)
- [28] Gentry, C., Halevi, S.: Implementing gentry's fully-homomorphic encryption scheme. In: EUROCRYPT 2011 (2011)
- [29] Gentry, C., Halevi, S., Peikert, C., Smart, N.P.: Ring switching in bgv-style homomorphic encryption. In: SCN (2012)
- [30] Gentry, C., Halevi, S., Smart, N.P.: Better bootstrapping in fully homomorphic encryption. In: PKC 2012 (2012)
- [31] Gentry, C., Halevi, S., Smart, N.P.: Fully homomorphic encryption with polylog overhead. In: Eurocrypt (2012)
- [32] Gentry, C., Halevi, S., Smart, N.P.: Homomorphic evaluation of the aes circuit. In: CRYPTO 2012 (2012)
- [33] Gentry, C., Sahai, A., Waters, B.: Homomorphic encryption from learning with errors: Conceptually-simpler, asymptotically-faster, attribute-based. In: Advances in Cryptology - CRYPTO 2013 - 33rd Annual Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2013. Proceedings, Part I (2013)
- [34] Goldwasser, S., Kalai, Y.T., Popa, R.A., Vaikuntanathan, V., Zeldovich, N.: Reusable garbled circuits and succinct functional encryption. In: Symposium on Theory of Computing Conference, STOC'13, Palo Alto, CA, USA. ACM (2013)
- [35] Gorbunov, S., Vaikuntanathan, V., Wichs, D.: Leveled fully homomorphic signatures from standard lattices. In: Proceedings of the forty-seventh annual ACM symposium on Theory of computing (2015)
- [36] Goyal, V., Li, H., Ostrovsky, R., Polychroniadou, A., Song, Y.: ATLAS: efficient and scalable MPC in the honest majority setting. In: Advances in Cryptology - CRYPTO 2021 - 41st Annual International Cryptology Conference, CRYPTO 2021, Virtual Event, Proceedings, Part II (2021)
- [37] Halevi, S., Polyakov, Y., Shoup, V.: An improved RNS variant of the BFV homomorphic encryption scheme. In: Topics in Cryptology - CT-RSA 2019 - The Cryptographers' Track at the RSA Conference (2019)

- [38] Halevi, S., Shoup, V.: Design and implementation of helib: a homomorphic encryption library. Cryptology ePrint Archive (2020)
- [39] Iliashenko, I., Zucca, V.: Faster homomorphic comparison operations for BGV and BFV. Proc. Priv. Enhancing Technol. (2021)
- [40] Kamara, S., Wei, L.: Garbled circuits via structured encryption. In: Adams, A.A., Brenner, M., Smith, M. (eds.) Financial Cryptography and Data Security - FC 2013 Workshops, USEC and WAHC 2013, Okinawa, Japan, April 1, 2013, Revised Selected Papers. Lecture Notes in Computer Science, vol. 7862, pp. 177–188. Springer (2013). https://doi.org/10.1007/978-3-642-41320-9_12, https://doi.org/10.1007/978-3-642-41320-9_12
- [41] Kamara, S., Wei, L.: Garbled circuits via structured encryption. In: Financial Cryptography and Data Security - FC 2013 Workshops, USEC and WAHC 2013, Okinawa, Japan (2013)
- [42] Mentens, N., Batina, L., Preneel, B., Verbauwhede, I.: A systematic evaluation of compact hardware implementations for the rijndael s-box. In: Topics in Cryptology–CT-RSA 2005 (2005)
- [43] Okamoto, T., Takashima, K.: Fully secure functional encryption with general relations from the decisional linear assumption. In: Annual cryptology conference (2010)
- [44] Patel, S., Persiano, G., Seo, J.Y., Yeo, K.: Efficient boolean search over encrypted data with reduced leakage. In: ASIACRYPT 2021 (2021)
- [45] Patranabis, S., Mukhopadhyay, D.: Forward and backward private conjunctive searchable symmetric encryption. In: NDSS 2021 (2021)
- [46] Smart, N.P.: Practical and efficient fhe-based MPC. In: Quaglia, E.A. (ed.) Cryptography and Coding - 19th IMA International Conference, IMACC 2023, London, UK, Proceedings (2023)
- [47] Song, D.X., Wagner, D., Perrig, A.: Practical techniques for searches on encrypted data. In: Proceeding 2000 IEEE symposium on security and privacy. S&P 2000 (2000)
- [48] Wang, Y., Sun, S.F., Wang, J., Chen, X., Liu, J.K., Gu, D.: Practical non-interactive encrypted conjunctive search with leakage suppression. In: Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security. p. 4658–4672. CCS '24, Association for Computing Machinery, New York, NY, USA (2024). <https://doi.org/10.1145/3658644.3670355>, <https://doi.org/10.1145/3658644.3670355>
- [49] Yang, K., Wang, X., Zhang, J.: More efficient MPC from improved triple generation and authenticated garbling. In: CCS '20: 2020 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, USA, 2020 (2020)
- [50] Yang, Q., Peng, G., Gasti, P., Balagani, K.S., Li, Y., Zhou, G.: MEG: memory and energy efficient garbled circuit evaluation on smartphones. IEEE Trans. Inf. Forensics Secur. (2019)
- [51] Yao, A.C.C.: How to generate and exchange secrets. In: 27th Annual Symposium on Foundations of Computer Science (sfcs 1986) (1986)

- [52] Yuan, B., Jia, Y., Xing, L., Zhao, D., Wang, X., Zou, D., Jin, H., Zhang, Y.: Shattered chain of trust: Understanding security risks in cross-cloud iot access delegation. In: USENIX Security Symposium (2020)
- [53] Zhou, W., Jia, Y., Yao, Y., Zhu, L., Guan, L., Mao, Y., Liu, P., Zhang, Y.: Discovering and understanding the security hazards in the interactions between iot devices, mobile apps, and clouds on smart home platforms. In: 28th USENIX Security Symposium (2019)